GLoBES.jl

General Long Baseline Experiment Simulator in Julia

User manual

Based on GLoBES 3.0 by P. Huber, J. Kopp, M. Lindner, M. Rolinec and W. Winter

Version 0.1 • October 18, 2026

About this manual

GLoBES.jl is a Julia implementation of GLoBES, the General Long Baseline Experiment Simulator [1, 2]. It simulates long-baseline and reactor neutrino oscillation experiments described in the Abstract Experiment Definition Language (AEDL), and computes event rates and $\Delta\chi^2$ functions with systematics, correlations and degeneracies. It reads the same AEDL files as GLoBES 3.0.

This manual follows the structure of the GLoBES 3.0 manual [3]. Part I explains how to use GLoBES.jl, with the physics and the formulas behind every computation. Part II describes AEDL. Part III is a reference of all functions, of the compatibility layer with the C function names, and of the places where GLoBES.jl deliberately differs from C GLoBES because the C code has a bug.

Every number and figure in this manual was computed with GLoBES.jl: the script `docs/manual/figures/make_figures.jl` produces the data of the figures, and the programs listed are in the `examples` directory of the package.

Conventions. Code is set in `typewriter`. Energies are in GeV, lengths in km, densities in g/cm^3, mass squared differences in eV^2 and angles in radians. All numbering (experiments, rules, channels, parameters) starts at 1. Boxes like the following point out differences from C GLoBES:

Difference from C GLoBES

In C GLoBES, experiments, rules and channels are numbered from 0. In GLoBES.jl they are numbered from 1, in the high-level API and in the `glb*` compatibility layer.

Contents

I The user's guide **1**

1 Introduction **2**
1.1 What GLoBES.jl does . . . 2
1.2 GLoBES.jl and GLoBES . . . 2
1.3 Installation . . . 3
1.4 How this manual is organised . . . 3

2 A tour of GLoBES.jl **4**
2.1 The program . . . 4
2.2 What it does . . . 5
2.3 The output . . . 6

3 Basic concepts **7**
3.1 Setups, experiments and simulated data . . . 7
3.2 Oscillation parameters . . . 7
3.3 Units and the integrated luminosity . . . 8
3.4 Simulated data . . . 8
3.5 Errors . . . 9
3.6 Several setups . . . 9

4 Oscillation probabilities **10**
4.1 Three-flavour oscillations in matter . . . 10
4.2 Computing probabilities . . . 11
4.3 Matter density profiles . . . 12
4.4 Low-pass filter* . . . 13

5 Event rates **14**
5.1 From fluxes to events . . . 14
5.2 Getting event rates . . . 14
5.3 Energy resolution . . . 15

6 χ^2 with systematics **17**
6.1 The Poisson χ^2 . . . 17
6.2 Systematics and the pull method . . . 17
6.2.1 Built-in systematics functions . . . 18
6.2.2 The energy calibration error . . . 19
6.3 Computing χ^2 . . . 19
6.4 User-defined systematics* . . . 20

7 Correlations, projections and degeneracies 22
7.1 Projections . . . 22
7.2 External information: priors . . . 22
7.3 Fitting . . . 22
7.4 Scans . . . 23
7.5 The minimiser . . . 23
7.6 Degeneracies . . . 24

8 Changing experiments at runtime 26
8.1 Exposure . . . 26
8.2 Baseline and matter profile . . . 26
8.3 Systematics and energy windows . . . 26
8.4 AEDL variables . . . 27
8.5 The low-pass filter* . . . 27

9 Non-standard physics 28

II The Abstract Experiment Definition Language 29

10 Getting started with AEDL 30
10.1 The concept . . . 30
10.2 A simple example . . . 30
10.3 Syntax . . . 32

11 Experiment definition with AEDL 34
11.1 Source and integrated luminosity . . . 34
11.2 Baseline and matter density profile . . . 35
11.3 Cross sections . . . 36
11.4 Channels . . . 36
11.5 Energy resolution . . . 36
11.6 Rules and systematics . . . 37
11.7 Testing AEDL files . . . 37

III Reference 38

12 Function reference 39

13 The `glb*` compatibility layer 41
13.1 Conventions . . . 41
13.2 Correspondence with the high-level API . . . 42
13.3 A translated program . . . 42

14 Differences from C GLoBES 44
14.1 Physical constants . . . 44
14.2 Flux tables . . . 44
14.3 Background tilt prior . . . 45
14.4 Empty bins in the Poisson χ^2 . . . 45
14.5 Energy scale shift . . . 45
14.6 The minimiser . . . 45
14.7 Data file lookup . . . 46
14.8 State and interface . . . 46

A Experiment files **47**

B Examples **48**

C Building this manual **49**

Part I

The user's guide

Chapter 1

Introduction

1.1 What GLoBES.jl does

A long-baseline neutrino experiment has a source of neutrinos (an accelerator beam, a neutrino factory, a beta beam or a nuclear reactor), a baseline of L kilometres through the Earth along which the neutrinos oscillate, and a detector that counts neutrinos of certain flavours in bins of reconstructed energy. GLoBES.jl computes

- three-flavour oscillation probabilities in matter of any density profile (Chapter 4);
- the expected event rates in every energy bin, for signals and backgrounds, including the energy resolution of the detector (Chapter 5);
- the $\Delta\chi^2$ of a hypothesis compared with simulated data, with systematic uncertainties (Chapter 6);
- the $\Delta\chi^2$ minimised over any set of oscillation parameters, with external information on them ("projections" or "marginalisation", Chapter 7), which is how GLoBES treats correlations and degeneracies.

Experiments are described in text files written in AEDL (Part II). Their flux, cross section, energy resolution, efficiency and systematics can be changed at runtime (Chapter 8), and the oscillation probabilities can be replaced by those of any non-standard model (Chapter 9).

1.2 GLoBES.jl and GLoBES

GLoBES.jl started as a line-by-line port of the C library GLoBES 3.0.11 and was tested against it: the test suite compares probabilities, event rates and χ^2 values of thirteen experiment setups with values computed by the C library. In the process, a number of bugs of the C library came to light. GLoBES.jl fixes them, so that some results differ from C GLoBES; Chapter 14 lists every difference and why it exists.

GLoBES.jl has two interfaces:

The high-level API (Chapters 2–9 and 12) uses named parameters (`OscParams(θ13=0.15, ...)`), experiment objects, keyword arguments, return values and exceptions. This is the interface to use for new programs.

The `glb*` functions (Chapter 13) have the names and arguments of the C functions, so that C GLoBES programs can be translated line by line. They work like the C library: errors are printed and the functions return error codes.

Both work on the same data and can be mixed.

1.3 Installation

GLoBES.jl needs Julia 1.10 or later. Its only dependency besides the standard library is `SpecialFunctions`. Install it from its directory with

```
julia> using Pkg
julia> Pkg.develop(path="/path/to/GLoBES.jl")
```

or start Julia with `julia --project=/path/to/GLoBES.jl`. Then

```
using GLoBES
```

loads it. The test suite is run with `julia --project=. test/runtests.jl` in the package directory.

Where AEDL files are found. `Experiment("NOvA.glb")` looks for the file in the current directory and then in the directories listed in the environment variable `GLB_PATH` (separated by `:`). Files that an AEDL file includes, and its flux and cross section files, are looked up in the current directory, next to the AEDL file, and in `GLB_PATH`. The experiments shipped with GLoBES.jl are in `test/data` and `examples`; Appendix A lists them.

1.4 How this manual is organised

Chapter 2 is a guided tour through a complete program. The following chapters explain each step in detail together with the formulas used. Sections marked with * describe advanced features that most users will not need.

Chapter 2

A tour of GLoBES.jl

This chapter goes through a complete program, `examples/tour.jl`. It simulates a neutrino factory with a baseline of 3000 km and asks how well it can tell the true value of $\sin^2 2\theta_{13} = 0.001$ from $\sin^2 2\theta_{13} = 0.0015$. The program is the GLoBES.jl version of the tour of the C GLoBES manual.

2.1 The program

```
using GLoBES
using Printf

cd(@__DIR__)                        # the AEDL file and its data files are here

# Oscillation parameters of the simulated data
truth = OscParams(θ12=asin(sqrt(0.8)) / 2, θ13=asin(sqrt(0.001)) / 2, θ23=π / 4,
                  δ=π / 2, Δm21=7e-5, Δm31=2e-3)

# A neutrino factory with a 3000 km baseline; simulate its data
nufact = Experiment("NFstandard.glb")
simulate!(truth)

# Oscillation probabilities ν_e → ν_e, ν_μ, ν_τ at 50 GeV
for f in (:e, :μ, :τ)
    @printf("P(νe → ν%s): %.6f in vacuum, %.6f in matter\n", f,
            probability(truth, :e, f; E=50, L=3000), probability(truth, :e, f, nufact;
             ↪ E=50))
end

# χ² of the hypothesis sin²2θ13 = 0.0015, minimised over systematics only
test = with_param(truth, :sin²2θ13, 0.0015)
@printf("\nχ² with systematics only:          %8.4f\n", chi2(test))
@printf("from the first rule only:          %8.4f\n", chi2(test; experiment=nufact,
 ↪ rule=1))
@printf("sum over the rules:                %8.4f\n",
        sum(chi2(test; rule=r) for r in eachindex(rule_names(nufact))))
@printf("statistics only:                   %8.4f\n", chi2(test; systematics=false))

# Correlations: minimise over all parameters except θ13, with 10% priors on
# the solar parameters and 5% on the matter density
priors = (θ12=0.1truth.θ12, Δm21=0.1truth.Δm21, ρ=0.05)
fit = marginalize(test; fixed=(:θ13,), priors)
@printf("\nχ² with correlations:              %8.4f\n", fit.chi2)
println("at the minimum: ", fit.params)
```

```
39
40 @printf("with δ fixed as well:               %8.4f\n",
41         marginalize(test; fixed=(:θ13, :δ), priors).chi2)
42 @printf("with only δ and ρ free:             %8.4f\n",
43         marginalize(test; free=(:δ, :ρ), priors).chi2)
44
45 # The sign degeneracy: start the fit at Δm31 < 0 and minimise over everything,
46 # with an additional 1/3 prior on |Δm31|
47 mirror = OscParams(truth; Δm31=-truth.Δm31)
48 deg = marginalize(mirror; priors=(priors..., Δm31=truth.Δm31 / 3), centers=mirror)
49 @printf("\nχ² at the inverted-hierarchy minimum: %6.4f\n", deg.chi2)
50 println("at ", deg.params)
51
52 # Two experiments: a second neutrino factory at the "magic" baseline
53 magic = Experiment("NFstandard.glb")
54 density_profile!(magic, average_profile(7500)...)
55 L, ρ = density_profile(magic)
56 @printf("\nmagic baseline: %g km, average density %.4f g/cm³\n", L[1], ρ[1])
57 simulate!(truth)
58 @printf("χ² of both experiments:            %8.4f\n", chi2(test))
59 @printf("χ² at 7500 km:                     %8.4f\n", chi2(test; experiment=magic))
60 fit = marginalize(test; fixed=(:θ13,), priors)
61 @printf("with correlations:                 %8.4f\n", fit.chi2)
```

2.2 What it does

Lines 6–8: the parameters. An `OscParams` holds the six oscillation parameters θ_{12}, θ_{13}, θ_{23}, δ_{CP}, Δm^2_{21} and Δm^2_{31}, and the matter density scaling factor ρ, which multiplies the density profile of each experiment and is 1 unless given. The parameters are named, so their order does not matter; ASCII names such as `theta13` and `dm31` work too.

Lines 10–12: the experiment and the simulated data. `Experiment` reads an AEDL file. `simulate!` computes the event rates of all loaded experiments for the true parameters. These rates are the "data" every χ^2 is computed against.

Lines 14–18: probabilities. `probability(params, from, to; E, L)` is the vacuum oscillation probability; with an experiment instead of a baseline it is computed along the experiment's matter density profile.

Lines 20–26: χ^2 with systematics. `with_param` changes a parameter given as $\sin^2 2\theta_{13}$; `chi2` compares the event rates predicted for the test parameters with the simulated ones. It is minimised over the systematic uncertainties ("nuisance parameters") of each rule of each experiment, but not over oscillation parameters. The rules of an experiment are its independent measurements, here the $\nu_e \to \nu_\mu$ appearance and the ν_μ disappearance channels of μ^+ and μ^- running. Their χ^2 add up to the total. `systematics=false` gives the purely statistical χ^2.

Lines 28–38: correlations. `marginalize` minimises χ^2 over oscillation parameters. Here all parameters except θ_{13} are free, which is the χ^2 of the hypothesis $\sin^2 2\theta_{13} = 0.0015$ "projected" onto the θ_{13} axis. The solar parameters cannot be measured by this experiment and get Gaussian priors (external information) of 10%, the matter density one of 5%. The fit finds a lower χ^2 because other parameters can partly compensate for the wrong value of θ_{13}. `fixed` and `free` choose which parameters are fitted.

Lines 40–45: degeneracies. The minimiser finds the local minimum nearest to its starting point. A fit started at $\Delta m^2_{31} < 0$ finds the *sign degeneracy*, the best fit with inverted mass ordering, and its χ^2 tells how well the experiment excludes it.

Lines 47–56: two experiments. A second neutrino factory gets a baseline of 7500 km (the "magic baseline" [13]) through `density_profile!`. The data are simulated again for both experiments, and `chi2` and `marginalize` now add the χ^2 of both. With `experiment=magic`, only one experiment is used.

2.3 The output

```
Precompiling packages...
  10437.7 ms  ✓ GLoBES
  1 dependency successfully precompiled in 11 seconds. 13 already precompiled.
P(νe → νe): 0.999955 in vacuum, 0.999965 in matter
P(νe → νμ): 0.000026 in vacuum, 0.000020 in matter
P(νe → ντ): 0.000019 in vacuum, 0.000015 in matter

χ² with systematics only:            21.6592
from the first rule only:            20.4451
sum over the rules:                  21.6592
statistics only:                     35.1847

χ² with correlations:                 2.0310
at the minimum: OscParams(θ12=0.54144, θ13=0.01937, θ23=0.74526, δ=1.7483, Δm21=6.6472e-05, Δm31=0.0020054, ρ=1.0
with δ fixed as well:                 4.0329
with only δ and ρ free:               2.7875

χ² at the inverted-hierarchy minimum: 4.5557
at OscParams(θ12=0.5861, θ13=0.027644, θ23=0.72443, δ=1.0473, Δm21=7.8378e-05, Δm31=-0.0019133, ρ=0.97732)

magic baseline: 7500 km, average density 4.2529 g/cm³
χ² of both experiments:              28.2240
χ² at 7500 km:                        6.5648
with correlations:                    3.5797
```

Systematics reduce the χ^2 from 35 (statistics only) to 22, and correlations with the other parameters, mainly δ_{CP}, reduce it to 2.0: on its own, the neutrino factory cannot distinguish $\sin^2 2\theta_{13} = 0.0015$ from 0.001 at 2σ once δ_{CP} is unknown (Table 7.1). With δ_{CP} fixed the χ^2 is 4.0. The inverted-ordering solution has $\chi^2 = 4.56$, so the mass ordering is determined at only about 2σ. The second detector at the magic baseline, where the oscillations hardly depend on δ_{CP}, raises the χ^2 with correlations to 3.6.

Chapter 3

Basic concepts

3.1 Setups, experiments and simulated data

A GLoBES.jl computation involves

A setup (`Setup`): a list of experiments, their simulated data, the probability engine, the registered χ^2 functions, and the minimiser settings. There is always a current setup, and most programs use only that one. Section 3.6 explains how to work with several.

Experiments (`Experiment`), each loaded from an AEDL file. An experiment has one neutrino source, one baseline and matter density profile, and one detector with one or more *rules*: independent measurements with their own signals, backgrounds and systematics.

Oscillation parameters (`OscParams`): the point in parameter space at which data are simulated or a hypothesis is tested.

The steps of an analysis are always the same:

```
using GLoBES
exp   = Experiment("T2K.glb")                          # 1. load experiments
truth = OscParams(θ12=0.5843, θ13=0.1503, θ23=0.8326,
                  δ=1.5π, Δm21=7.53e-5, Δm31=2.5e-3)
simulate!(truth)                                       # 2. simulate the data
chi2(OscParams(truth; δ=0))                            # 3. test hypotheses
```

3.2 Oscillation parameters

The standard parameters are listed in Table 3.1. They follow the standard parametrisation of the leptonic mixing matrix,

$$U = \begin{pmatrix} 1 & 0 & 0 \\ 0 & c_{23} & s_{23} \\ 0 & -s_{23} & c_{23} \end{pmatrix} \begin{pmatrix} c_{13} & 0 & s_{13}e^{-i\delta_{\mathrm{CP}}} \\ 0 & 1 & 0 \\ -s_{13}e^{i\delta_{\mathrm{CP}}} & 0 & c_{13} \end{pmatrix} \begin{pmatrix} c_{12} & s_{12} & 0 \\ -s_{12} & c_{12} & 0 \\ 0 & 0 & 1 \end{pmatrix}, \tag{3.1}$$

with $s_{ij} = \sin\theta_{ij}$, $c_{ij} = \cos\theta_{ij}$, and $\Delta m^2_{ij} = m^2_i - m^2_j$. The mass ordering is normal for $\Delta m^2_{31} > 0$ and inverted for $\Delta m^2_{31} < 0$.

Creating parameters. All of θ_{12}, θ_{13}, θ_{23}, Δm^2_{21} and Δm^2_{31} must be given; δ_{CP} defaults to 0 and ρ to 1:

```
p = OscParams(θ12=asin(sqrt(0.307)), θ13=asin(sqrt(0.022)), θ23=asin(sqrt(0.546)),
              δ=1.5π, Δm21=7.53e-5, Δm31=2.5e-3)
q = OscParams(p; δ=0, Δm31=-2.4e-3)       # a copy with some parameters changed
r = with_param(p, :sin²2θ13, 0.085)        # set θ13 through sin²2θ13
p.θ23, p.sin²θ23, p.ρ                      # read parameters
```

Parameter	Name	ASCII name	Unit	C index
θ_{12}	`θ12`	`theta12`	rad	`GLB_THETA_12`
θ_{13}	`θ13`	`theta13`	rad	`GLB_THETA_13`
θ_{23}	`θ23`	`theta23`	rad	`GLB_THETA_23`
δ_{CP}	`δ`	`delta`	rad	`GLB_DELTA_CP`
Δm^2_{21}	`Δm21`	`dm21`	eV^2	`GLB_DM_21`
Δm^2_{31}	`Δm31`	`dm31`	eV^2	`GLB_DM_31`
matter density scaling	`ρ`	`rho`	1	density parameters
$\sin^2\theta_{12}$, $\sin^2\theta_{13}$, $\sin^2\theta_{23}$	`sin²θ12` …	`sin2theta12` …		
$\sin^2 2\theta_{12}$, $\sin^2 2\theta_{13}$, $\sin^2 2\theta_{23}$	`sin²2θ12` …	`sin22theta12` …		

Table 3.1: Names of the oscillation parameters. The last two rows are derived quantities, which can be read (`p.sin²2θ13`) and set with `with_param` and in scans, but are not stored.

`OscParams` are immutable: functions never change them, and a changed copy is made with `OscParams(p; changes...)` or `with_param`.

Matter density. `ρ` scales the matter density profile of the experiments: the densities of the AEDL file are multiplied by ρ when rates are computed. It is one number for all experiments, or a vector with one entry per experiment, `OscParams(p; ρ=[1.0, 0.97])`. Its uncertainty is taken into account by fitting it with a prior (Section 7.2).

Additional parameters. A probability engine for non-standard physics (Chapter 9) may have more parameters. They are given as a named tuple, `OscParams(p; extra=(ε=0.1,))`, and accessed by name like the standard ones.

3.3 Units and the integrated luminosity

GLoBES.jl uses GeV for neutrino energies, km for baselines, $\mathrm{g/cm}^3$ for matter densities, eV^2 for mass squared differences and radians for angles. The number of events is proportional to the *integrated luminosity*,

$$N \propto \text{source power} \times \text{running time} \times \text{target mass}, \tag{3.2}$$

with units chosen in the AEDL file: typically MW of beam power (or useful parent decays per year for neutrino factories and beta beams), years, and kt (t for reactor detectors). Section 11.1 explains how the normalisation of the flux tables relates these units to event numbers. At runtime, they are changed with `source_power!`, `running_time!` and `target_mass!` (Section 8.1).

3.4 Simulated data

```
simulate!(truth::OscParams; setup=current_setup())
```

Compute the event rates of all experiments of the setup for the parameters `truth`. They serve as the data that all χ^2 values compare to.

The simulated data are the event rates expected for the true parameters; they are not fluctuated. This gives the $\Delta\chi^2$ of an *average* experiment (the "Asimov data set"): the expected sensitivity. Since the true parameters fit the data perfectly, $\chi^2_{\mathrm{min}} = 0$ and χ^2 and $\Delta\chi^2$ are the same.

`simulate!` must be called again after experiments are added or changed; `chi2` and `marginalize` throw an error if no data have been simulated. `simulated_params()` returns the parameters of the last call.

3.5 Errors

The functions of the high-level API throw a `GLoBESError` when something is wrong: a file that cannot be read, an unknown rule name, a missing `simulate!`, a parameter vector of the wrong length. Errors in AEDL files are `AEDLError`s with the file name and line. The `glb*` functions instead print errors and return error codes, like the C library; their output is controlled with `glbSetVerbosityLevel`.

3.6 Several setups

Each `Setup` is an independent simulation. Every function works on the current setup unless it gets a `setup` keyword or an experiment, which belongs to the setup it was loaded into:

```
near = Setup()
far  = Setup()
Experiment("NOvA.glb"; setup=near)
Experiment("NOvA.glb"; setup=far, BASELINE=1500)   # BASELINE: an AEDL variable
simulate!(truth; setup=near); simulate!(truth; setup=far)
chi2(test; setup=near), chi2(test; setup=far)
```

`with_setup(f, setup)` makes `setup` the current setup while `f` runs, which is convenient for longer pieces of code and also applies to the `glb*` functions:

```
with_setup(far) do
    simulate!(truth)
    glbChiSys(glbparams, GLB_ALL, GLB_ALL)   # the glb* layer sees `far` too
end
```

`current_setup()` returns the current setup.

Note

GLoBES.jl is not thread-safe: functions make their setup the current one while they run, so two threads computing at the same time interfere even if they use different setups. To compute in parallel, use several processes (for instance with the `Distributed` standard library), each with its own experiments.

Difference from C GLoBES

C GLoBES keeps all its state (experiments, rates, minimiser settings, the probability engine) in global variables, so a program can only have one simulation. In GLoBES.jl, this state belongs to a `Setup`.

Chapter 4

Oscillation probabilities

4.1 Three-flavour oscillations in matter

A neutrino of energy E produced in flavour α evolves in matter of constant electron density N_e with the Hamiltonian (in flavour basis)

$$H = \frac{1}{2E}\, U \begin{pmatrix} 0 & & \\ & \Delta m^2_{21} & \\ & & \Delta m^2_{31} \end{pmatrix} U^\dagger + \begin{pmatrix} V & & \\ & 0 & \\ & & 0 \end{pmatrix}, \qquad V = \sqrt{2}\, G_F N_e, \tag{4.1}$$

with U from Eq. (3.1). The matter potential V comes from coherent forward scattering on electrons. For antineutrinos, $U \to U^*$ (that is, $\delta_{\mathrm{CP}} \to -\delta_{\mathrm{CP}}$) and $V \to -V$. The evolution through a layer of length L is

$$S(L) = e^{-iHL} = Q\, \mathrm{diag}(e^{-i\lambda_1 L}, e^{-i\lambda_2 L}, e^{-i\lambda_3 L})\, Q^\dagger, \tag{4.2}$$

where λ_k and the columns of Q are the eigenvalues and eigenvectors of H. GLoBES.jl diagonalises H with the fast algorithms for Hermitian 3×3 matrices of Ref. [4]. A matter profile is a sequence of layers of constant density with lengths L_k and densities ρ_k. The evolution through it is the product of the layers' evolution operators, and the probability is

$$P(\nu_\alpha \to \nu_\beta) = \left| \left[S(L_n) \cdots S(L_2)\, S(L_1) \right]_{\beta\alpha} \right|^2. \tag{4.3}$$

In vacuum ($V = 0$) the eigenvectors are the columns of U, and Eq. (4.3) reduces to the familiar

$$P(\nu_\alpha \to \nu_\beta) = \delta_{\alpha\beta} - 4 \sum_{i>j} \mathrm{Re}(U^*_{\alpha i} U_{\beta i} U_{\alpha j} U^*_{\beta j}) \sin^2 \frac{\Delta m^2_{ij} L}{4E} + 2 \sum_{i>j} \mathrm{Im}(U^*_{\alpha i} U_{\beta i} U_{\alpha j} U^*_{\beta j}) \sin \frac{\Delta m^2_{ij} L}{2E}. \tag{4.4}$$

Units. With Δm^2 in eV^2, L in km and E in GeV, the oscillation phase is

$$\frac{\Delta m^2 L}{4E} = \frac{\Delta m^2\,[\mathrm{eV}^2]\; L\,[\mathrm{km}]}{4\,\hbar c\; E\,[\mathrm{eV}]} \approx 1.267\, \frac{\Delta m^2\,[\mathrm{eV}^2]\; L\,[\mathrm{km}]}{E\,[\mathrm{GeV}]}, \qquad \hbar c = 1.973\,269\,804 \times 10^{-10}\,\mathrm{eV\,km}. \tag{4.5}$$

The matter potential is computed from the density as

$$V = \sqrt{2}\, G_F N_A\, Y_e\, \rho \approx 3.82 \times 10^{-14}\,\mathrm{eV} \times \frac{\rho}{\mathrm{g/cm^3}}, \qquad Y_e = \tfrac{1}{2}, \tag{4.6}$$

with the Fermi constant $G_F = 1.166\,3787 \times 10^{-5}\,\mathrm{GeV}^{-2}$, Avogadro's number N_A and Y_e electrons per nucleon, the value for the Earth's mantle and crust.

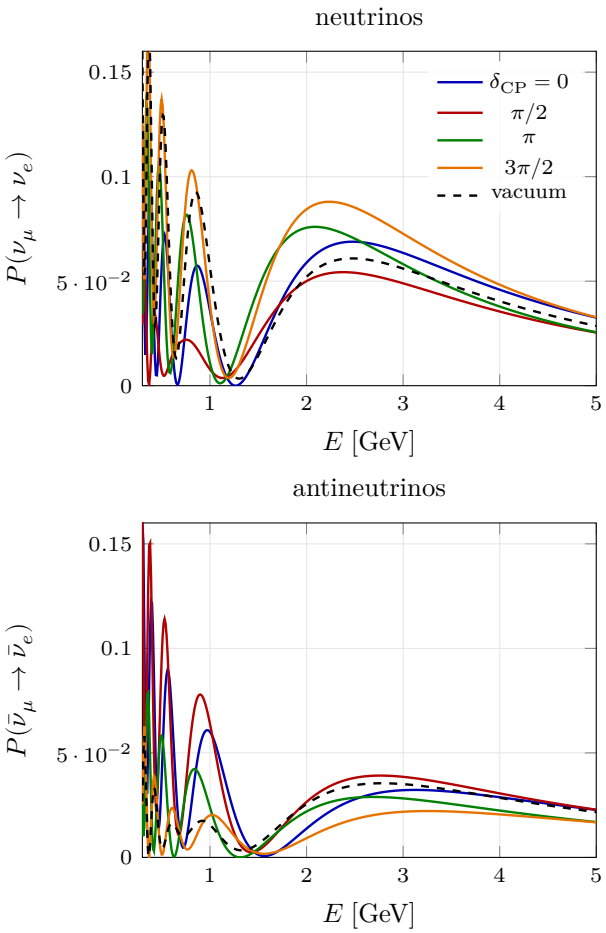


Figure 4.1: Appearance probabilities at $L = 1300\,\mathrm{km}$ in matter of constant density $\rho = 2.8\,\mathrm{g/cm^3}$ for four values of δ_{CP} and, dashed, in vacuum for $\delta_{\mathrm{CP}} = 3\pi/2$. The parameters are $\sin^2\theta_{12} = 0.307$, $\sin^2\theta_{13} = 0.022$, $\sin^2\theta_{23} = 0.546$, $\Delta m^2_{21} = 7.53 \times 10^{-5}\,\mathrm{eV}^2$, $\Delta m^2_{31} = 2.5 \times 10^{-3}\,\mathrm{eV}^2$. Matter enhances neutrino and suppresses antineutrino appearance for normal ordering. Computed with `probability(p, :μ, :e; E, L=1300, ρ=2.8)`.

Difference from C GLoBES

C GLoBES uses $\hbar c = 1.974\,7235 \times 10^{-10}\,\mathrm{eV\,km}$ and $\sqrt{2}G_F N_A = 7.5 \times 10^{-14}\,\mathrm{eV}$ per $\mathrm{g/cm^3}$, which are 0.07% and 1.7% away from the physical values. GLoBES.jl uses the values above (Section 14.1).

4.2 Computing probabilities

```
probability(params, from, to; E, L, ρ=0, antineutrino=false)
probability(params, from, to, experiment; E, antineutrino=false)
```

$P(\nu_{\mathrm{from}} \to \nu_{\mathrm{to}})$ for the oscillation parameters `params` at energy `E` (GeV; a number, or a vector or range for many energies), over `L` km of constant density `ρ` ($\mathrm{g/cm^3}$; vacuum for $\rho = 0$), or along the baseline and density profile of an experiment. Flavours are `:e`, `:μ`, `:τ` (also `:mu`, `:tau`, or 1, 2, 3).

Figure 4.1 shows appearance probabilities computed this way. The ρ of `params` is not used, since the density is given:

```
E = range(0.3, 5, 471)
```

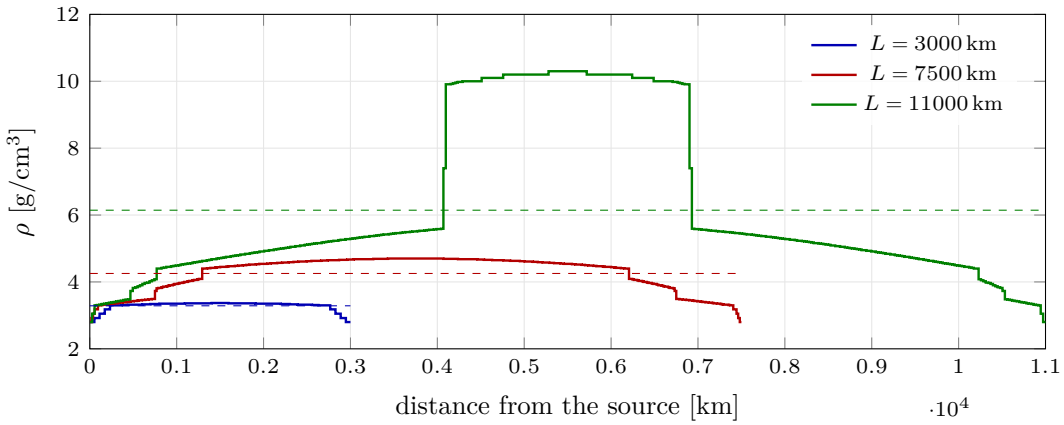


Figure 4.2: The matter density along baselines of 3000, 7500 and 11000 km (`prem_profile(L, 400)`), and the average densities (dashed, `average_profile(L)`). The longest baseline crosses the Earth's outer core.

```
P = probability(truth, :μ, :e; E, L=1300, ρ=2.8)
P̄ = probability(truth, :μ, :e; E, L=1300, ρ=2.8, antineutrino=true)
probability(truth, :μ, :μ, nova; E=2.0)                    # along NOvA's profile
```

4.3 Matter density profiles

Every experiment has a matter density profile along its baseline, chosen in its AEDL file (Section 11.2). The Earth's density is taken from the Stacey model [6], a tabulation of the density $\rho(r)$ in 500 shells, close to PREM [5]. A neutrino travelling a baseline L from a point on the surface reaches the distance

$$r(x) = \sqrt{x^2 + R_\oplus^2 - Lx} \tag{4.7}$$

from the centre of the Earth after a distance x, with $R_\oplus = 6371\,\text{km}$. A profile of n layers has the lengths L/n and the densities $\rho(r(x_k))$ at the layer centres x_k; the average-density profile is a single layer with the mean of 1000 such layers. Figure 4.2 shows profiles for three baselines.

```
prem_profile(L, layers) -> (lengths, densities)
average_profile(L) -> (lengths, densities)
```

The Earth's density along a baseline of `L` km, in `layers` layers of equal length or as a single layer of average density.

```
density_profile(e) -> (lengths, densities)
density_profile!(e, lengths, densities)
```

The profile of an experiment, and replacing it by any profile. The baseline becomes the sum of the lengths.

```
baseline(e)    baseline!(e, L)
```

The baseline of an experiment. Changing it recomputes the profile in the way the AEDL file defines it: average density, n Stacey layers, or the user layers scaled to the new length.

For most experiments, one layer with the average density and a 5% uncertainty on it (Section 7.2) is accurate enough [8].

4.4 Low-pass filter*

For high frequencies, fast oscillations average out within an energy bin but cause aliasing when sampled at a finite number of energies. The low-pass filter damps the oscillation terms of the probability with the phase $\phi_{ij} = (\lambda_i - \lambda_j)L$ by

$$P(\nu_\alpha \to \nu_\beta) = \sum_i |Q_{\alpha i}|^2 |Q_{\beta i}|^2 + 2 \sum_{i<j} \mathrm{Re}\Big(Q_{\alpha j} Q^*_{\beta j} Q^*_{\alpha i} Q_{\beta i}\, e^{-i\phi_{ij}} \Big) \exp\Big(-\frac{\phi_{ij}^2\, \sigma_E^2}{2E^2} \Big), \tag{4.8}$$

which is the probability averaged over a Gaussian energy distribution of width σ_E in the approximation $\phi \propto 1/E$. It is switched on with `$filter_state = 1` and `$filter_value =` σ_E in GeV in AEDL (Section 11.5) and only works for a single-layer profile. The filter also reduces the width of the energy resolution function so that the total smearing stays the same (Section 5.3).

Chapter 5

Event rates

5.1 From fluxes to events

GLoBES computes events in two energy grids: the *sampling points* E_j (with widths ΔE_j) in true neutrino energy, and the *bins* i in reconstructed energy (with edges $E_i^{\pm}$, centres E_i and widths ΔE_i). A *channel* describes neutrinos produced as ν_α (or $\bar{\nu}_\alpha$) in a flux, detected as ν_β through a cross section, with an energy resolution (Figure 5.1). The number of events of channel c at sampling point j is

$$n_{c,j} = \Phi_\alpha(E_j)\, P(\nu_\alpha \to \nu_\beta; E_j)\, \sigma_\beta(E_j)\, M\, \epsilon^{\text{pre}}_{c,j}\, \Delta E_j + B^{\text{pre}}_{c,j}, \tag{5.1}$$

with the detector mass M and the pre-smearing efficiencies and backgrounds ϵ^{pre} (default 1) and B^{pre} (default 0). The flux at the detector is

$$\Phi_\alpha(E) = \frac{\mathcal{N}}{L^2}\, \texttt{@norm} \times \texttt{@power} \times \texttt{@stored_muons} \times \texttt{@time}\, \phi_\alpha(E), \tag{5.2}$$

where ϕ_α is the flux table of the AEDL file (or a built-in flux), normalised to a distance of 1 km, and $\mathcal{N}$ a constant that depends on the kind of flux (Section 11.1). The cross section $\sigma_\beta(E) = E\,(\sigma/E)_\beta(E)$ is in $10^{-38}\,\text{cm}^2$ per nucleon. Channels whose flavours are prefixed with `NOSC_` in AEDL do not oscillate: $P = \delta_{\alpha\beta}$.

The energy resolution maps the sampling points to the bins,

$$N_{c,i} = \epsilon^{\text{post}}_{c,i} \sum_j K^{(c)}_{ij}\, n_{c,j} + B^{\text{post}}_{c,i}, \tag{5.3}$$

with the smearing matrix K_{ij}, the probability that an event at sampling point j is reconstructed in bin i (Section 5.3), and post-smearing efficiencies and backgrounds.

Rules. A rule adds channels with coefficients to a signal and a background,

$$s_i = \sum_k a_k\, N_{c_k,i}, \qquad b_i = \sum_k b_k\, N_{c'_k,i}, \tag{5.4}$$

and the prediction without systematics is $s_i + \beta_0 b_i$ with the background normalisation β_0 given by `@backgroundcenter`. Only the bins in the energy window of the rule enter its χ^2.

5.2 Getting event rates

```
event_rates(e; rule=1)
event_rates(e, params::OscParams; rule=1)
```

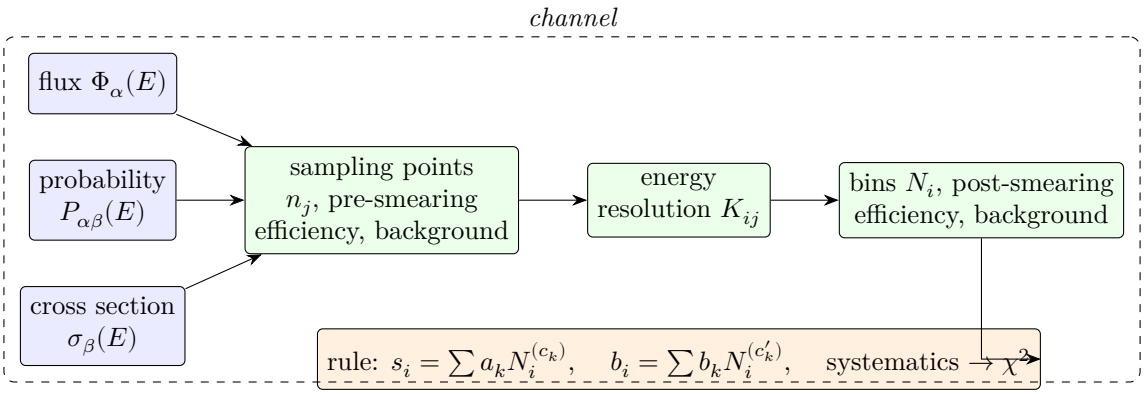


Figure 5.1: How GLoBES computes event rates. A *channel* turns the flux of one neutrino flavour into events of one flavour in the detector, binned in reconstructed energy; a *rule* combines channels into signal and background and computes χ^2.

The rates of one rule in each bin: of the simulated data, or as predicted for `params`. Returns a named tuple `(energy, width, signal, background, total)`: bin centres and widths, s_i, $\beta_0 b_i$, and $s_i + \beta_0 b_i$ inside the energy window (zero outside). `rule` is a number or the name of the rule in the AEDL file.

```
total_events(e; rule=1)   total_events(e, params; rule=1)
```

The sum of `event_rates(...).total`.

```
channel_rates(e, channel; smeared=true)
```

The simulated $N_{c,i}$ of a channel per bin, or with `smeared=false` $n_{c,j}$ per sampling point.

```
rule_names(e)   channel_names(e)   bin_centers(e)   bin_widths(e)   energy_range(e)
```

Names of rules and channels as in the AEDL file (without `#`), and the reconstructed energy binning.

```
energy_window(e; rule=1)   energy_window!(e, emin, emax; rule=:all)
energy_window_bins(e; rule=1)
```

The energy window of a rule in GeV, changing it, and the bins inside it.

For example, the spectra of Figure 5.2 come from

```
nova = Experiment("NOvA.glb")
simulate!(truth)
r  = event_rates(nova; rule="Nu_E_Appearance")
io = event_rates(nova, OscParams(truth; Δm31=-truth.Δm31 + truth.Δm21); rule=1)
r.energy, r.signal, r.background, sum(r.total), sum(io.total)
```

5.3 Energy resolution

The energy resolution of a channel is either given as a matrix in the AEDL file or computed from a resolution function (Section 11.5). For a Gaussian resolution with width $\sigma(E)$, the element K_{ij} is the probability that an event with true energy E_j is reconstructed in bin i,

$$K_{ij} = \frac{1}{2}\left[\operatorname{erf}\left(\frac{E_i^+ - E_j}{\sqrt{2}\,\sigma(E_j)}\right) - \operatorname{erf}\left(\frac{E_i^- - E_j}{\sqrt{2}\,\sigma(E_j)}\right)\right], \tag{5.5}$$

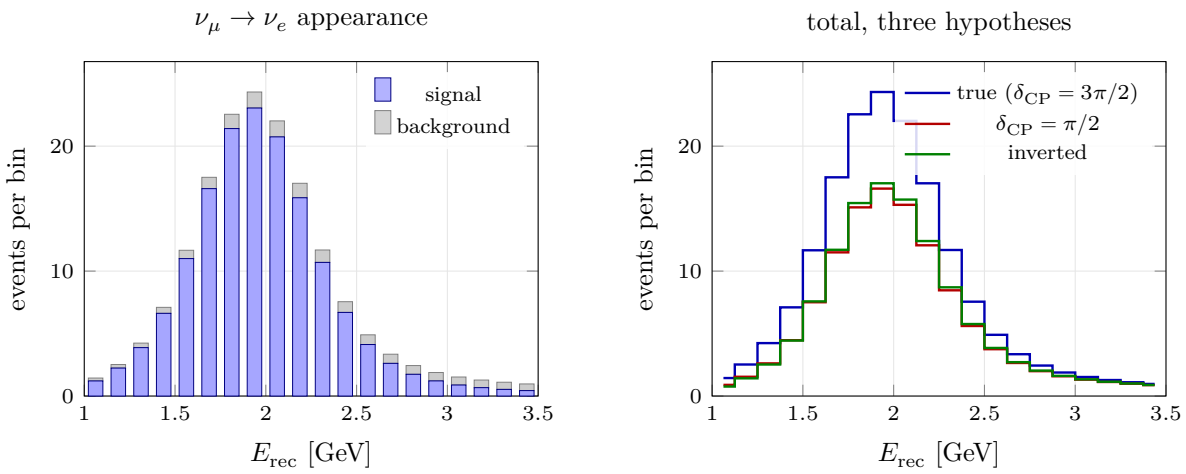


Figure 5.2: The ν_e appearance rule of `NOvA.glb`. Left: signal and background of the simulated data. Right: the total rates for the true parameters, for $\delta_{\text{CP}} = \pi/2$, and for inverted ordering ($\Delta m^2_{31} \to -\Delta m^2_{31} + \Delta m^2_{21}$). Parameters as in Figure 4.1.

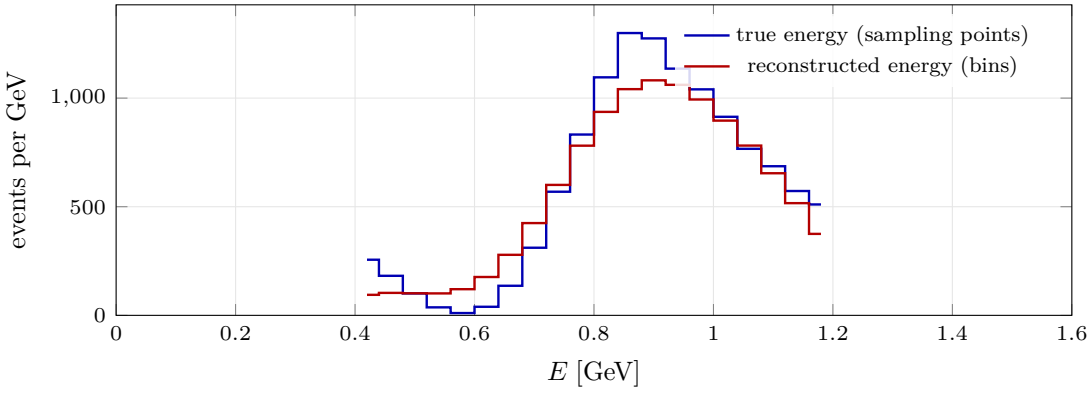


Figure 5.3: The ν_e appearance signal channel of `T2K.glb` before and after the energy resolution, $\sigma_E = 85\,\text{MeV}$. Rates are divided by the widths of sampling points and bins. Computed with `channel_rates(t2k, "nu_e_signal"; smeared=false)` and `channel_rates(t2k, "nu_e_signal")`.

rounded to multiples of 10^{-7}, so that the matrix is sparse. The built-in resolution functions are

$$\texttt{\#standard:}\quad \sigma(E) = a\,E + b\sqrt{E} + c, \qquad \texttt{@sigma_e = \{a, b, c\}}, \tag{5.6}$$

$$\texttt{\#inverse_beta:}\quad \sigma(E) = \alpha\sqrt{E/\text{MeV} - 0.8}\,\text{MeV}, \qquad \texttt{@sigma_e = \{α\}}, \tag{5.7}$$

the second for reactor antineutrinos detected by inverse beta decay, whose visible energy is $E - 0.8\,\text{MeV}$ (it is $\alpha\,\text{keV}$ below $1.8\,\text{MeV}$). With the low-pass filter (Section 4.4) and `@type = 2`, the width is reduced to $\sqrt{\sigma(E)^2 - \sigma_f^2}$, since the filter already averages over σ_f. Figure 5.3 shows a spectrum before and after smearing.

Events whose reconstructed energy falls outside the bins are lost. The sampling points should therefore extend beyond the bins by a few σ (`$sampling_min`, `$sampling_max`); by default they coincide with the bins.

Chapter 6

χ^2 with systematics

6.1 The Poisson χ^2

GLoBES compares the simulated event numbers n_i (the "data", from `simulate!`) with the event numbers N_i predicted for a hypothesis with the Poisson log-likelihood ratio

$$\chi^2 = \sum_i \chi^2_{\mathrm{P}}(n_i, N_i), \qquad \chi^2_{\mathrm{P}}(n, N) = 2\Big(N - n + n \ln \frac{n}{N}\Big), \tag{6.1}$$

which is $-2\ln$ of the ratio of the Poisson probabilities of n for the predicted mean N and for the best possible mean n. For large event numbers, $\chi^2_{\mathrm{P}} \to (n-N)^2/n$, the Gaussian χ^2 (Figure 6.1). The function is also available as `poisson_chi2(n, N)`, for user-defined χ^2 functions.

Empty bins and vanishing predictions. For $n = 0$, $\chi^2_{\mathrm{P}} = 2N$; an empty bin with $N = 0$ contributes nothing. If systematics push the prediction to $N \le 0$ while $n > 0$, the logarithm diverges. Below $N_0 = 10^{-10}$, χ^2_{P} is therefore continued by its Taylor expansion of second order around N_0, a steep but finite and smooth wall that minimisers can handle:

$$\chi^2_{\mathrm{P}}(n, N) = \chi^2_{\mathrm{P}}(n, N_0) + 2\Big(1 - \frac{n}{N_0}\Big)(N - N_0) + \frac{n}{N_0^2}(N - N_0)^2 \qquad (N < N_0). \tag{6.2}$$

A negative prediction for an empty bin gives $2|N|$.

Difference from C GLoBES

C GLoBES returns 10^{100} for every bin with $N \le 0$, including empty bins with $n = N = 0$. A rule with an empty bin (for instance a background that vanishes at some energies) then has $\chi^2 = 10^{100}$ whatever the parameters, and minimisers see a discontinuous function (Section 14.4).

6.2 Systematics and the pull method

Systematic uncertainties are described by nuisance parameters ζ_k (the "pulls"): for example, the signal normalisation is $(1 + \zeta_1)$ times the nominal one, with a Gaussian uncertainty σ_1. The χ^2 of a rule is minimised over them,

$$\chi^2_{\mathrm{rule}} = \min_{\zeta} \Big[\sum_{i \in \mathrm{window}} \chi^2_{\mathrm{P}}(n_i, N_i(\zeta)) + \sum_k \frac{(\zeta_k - \zeta_k^0)^2}{\sigma_k^2} \Big], \tag{6.3}$$

independently for every rule [7], and the χ^2 of an experiment is the sum over its rules. The minimisation uses Powell's method (Section 7.5).

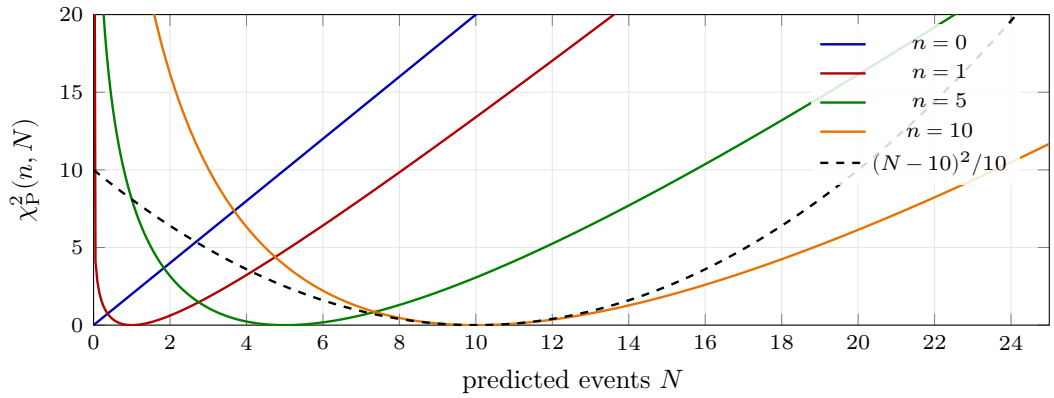


Figure 6.1: The Poisson χ^2 of one bin with n observed (simulated) events as a function of the prediction N, and the Gaussian approximation for $n = 10$.

Name	Parameters	Old `errordim`	Description
`chiSpectrumTilt`	4	0	normalisation and tilt of signal and background, Eq. (6.4)
`chiNoSysSpectrum`	0	2	spectrum without systematics
`chiTotalRatesTilt`	4	4	as `chiSpectrumTilt`, total rates only
`chiSpectrumOnly`	1	7	spectrum with free signal normalisation
`chiNoSysTotalRates`	0	8	total rates without systematics
`chiSpectrumCalib`	4	9	normalisation and energy calibration, Eq. (6.6)
`chiZero`	0	–	always 0 (rules used by other rules' χ^2)

Table 6.1: Built-in χ^2 functions. The old `errordim` numbers of GLoBES 2.x are still accepted in AEDL (`@errordim_sys_on`).

6.2.1 Built-in systematics functions

Each rule uses one χ^2 function with systematics on and one with systematics off (`@sys_on_function` and `@sys_off_function` in AEDL). The built-in functions are listed in Table 6.1. They use the signal s_i and background b_i of Eq. (5.4), the background normalisation β_0 and background tilt centre τ_0 of `@backgroundcenter` (default $1 : 0$), and the errors $\sigma_1, \ldots, \sigma_4$ (`@signalerror` $= \sigma_1 : \sigma_2$, `@backgrounderror` $= \sigma_3 : \sigma_4$, or `@sys_on_errors`). With $\bar{E} = (E_{\min} + E_{\max})/2$ and $\Delta E = E_{\max} - E_{\min}$ the reconstructed energy range of the experiment, and E_i the bin centres:

`chiSpectrumTilt` (four nuisance parameters):

$$N_i = (1 + \zeta_1)\, s_i + \frac{\zeta_2}{\Delta E}(E_i - \bar{E})\, s_i + \beta_0(1 + \zeta_3)\, b_i + \frac{\zeta_4}{\Delta E}(E_i - \bar{E})\, b_i, \tag{6.4}$$

$$\chi^2 = \sum_i \chi^2_P(n_i, N_i) + \frac{\zeta_1^2}{\sigma_1^2} + \frac{\zeta_2^2}{\sigma_2^2} + \frac{\beta_0^2\zeta_3^2}{\sigma_3^2} + \frac{(\zeta_4 - \tau_0)^2}{\sigma_4^2}. \tag{6.5}$$

The "tilt" ζ_2 is a linear distortion of the spectrum, a simple model of an energy calibration uncertainty.

`chiTotalRatesTilt`: the same N_i and priors, but with $\chi^2_P(\sum_i n_i, \sum_i N_i)$, so that only the total number of events is used.

`chiNoSysSpectrum`, `chiNoSysTotalRates`: $N_i = s_i + \beta_0 b_i$ without nuisance parameters, binned or total.

chiSpectrumOnly: $N_i = (1+\zeta_1)s_i + \beta_0 b_i$ with no prior on ζ_1: only the shape of the spectrum is used.

chiSpectrumCalib (energy calibration error):

$$N_i = (1+\zeta_1)\,\mathcal{S}_{\zeta_2}[s]_i + \beta_0(1+\zeta_3)\,\mathcal{S}_{\tau_0+\zeta_4}[b]_i, \tag{6.6}$$

$$\chi^2 = \sum_i \chi^2_{\mathrm{P}}(n_i, N_i) + \frac{\zeta_1^2}{\sigma_1^2} + \frac{\zeta_2^2}{\sigma_2^2} + \frac{\beta_0^2\zeta_3^2}{\sigma_3^2} + \frac{\zeta_4^2}{\sigma_4^2}, \tag{6.7}$$

where $\mathcal{S}_b$ shifts the energy scale of a spectrum by a relative amount b (Section 6.2.2).

Difference from C GLoBES

In `chiSpectrumTilt` and `chiTotalRatesTilt` C GLoBES applies the prior of the background tilt to $\zeta_4/\Delta E$ instead of ζ_4, so that the background tilt error is effectively $\sigma_4\,\Delta E$ instead of σ_4 (Section 14.3).

6.2.2 The energy calibration error

An energy scale error b means that an event reconstructed at energy E in the simulation is reconstructed at $(1+b)E$ in the real detector, or, equivalently, that the bin $[E_i^-, E_i^+]$ of the shifted spectrum contains the events of the nominal spectrum between $(1+b)E_i^-$ and $(1+b)E_i^+$. With the cumulative event number $F(E)$ of the nominal spectrum, which is known at the bin edges,

$$F(E_i^+) = \sum_{k\le i} s_k, \qquad \mathcal{S}_b[s]_i = F\big((1+b)E_i^+\big) - F\big((1+b)E_i^-\big). \tag{6.8}$$

Between the edges, F is interpolated with a monotone cubic Hermite interpolant (Fritsch–Carlson with the Fritsch–Butland slopes [11] and shape-preserving one-sided slopes at the ends). There are no events outside the binned range: F is 0 below it and constant above it. As a result, $\mathcal{S}_b$

- reproduces the spectrum for $b = 0$ and conserves the number of events inside the range,
- never gives negative rates, because F is monotone,
- is smooth in b, so that the minimisation over b is well behaved,
- works for bins of any width.

`energy_scale_shift(e, rates, b)` applies it to any spectrum of an experiment.

Difference from C GLoBES

C GLoBES computes $\mathcal{S}_b[s]_i = (1+b)\,s(\delta_i)$ by linear interpolation of the rates at the shifted bin positions $\delta_i = b(i + t_0 + \frac{1}{2}) + i$. Interpolating the rates and not their integral makes χ^2 kinked at $b = 0$ in every bin (Figure 6.2), the last bin is counted twice when the spectrum is shifted past the upper end, and the method requires equal bins. See Section 14.5.

6.3 Computing χ^2

```
chi2(test::OscParams; experiment=:all, rule=:all, systematics=true,
setup=current_setup())
```

χ^2 of the hypothesis `test`, minimised over the systematics of every rule but not over oscillation parameters, summed over all experiments and rules, or for one experiment (an `Experiment`) or one rule (a number or name; with `experiment=:all` the rule number in every experiment). `systematics=false` uses the `@sys_off_function` of the rules.

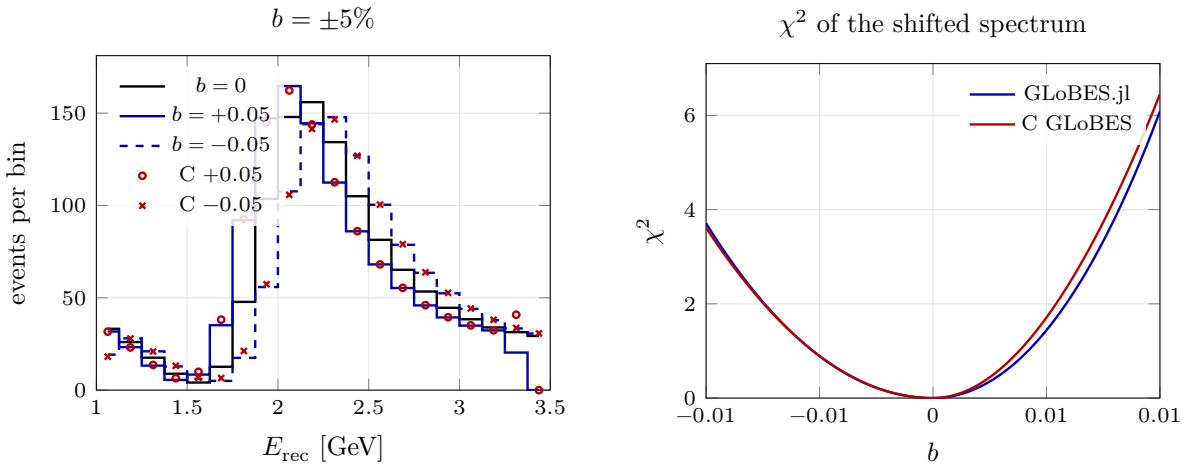


Figure 6.2: The energy scale shift applied to the ν_μ disappearance signal of `NOvA.glb`. Left: the spectrum shifted by $\pm 5\%$ with Eq. (6.8) (lines) and with the method of C GLoBES (symbols). Right: χ^2_{P} of the shifted against the nominal spectrum. The interpolation of C GLoBES makes it kinked at $b = 0$; Eq. (6.8) is smooth with a minimum at $b = 0$.

```
systematics!(e, on::Bool; rule=:all)    systematics!(on::Bool; setup)
```

Switch systematics on or off, for some rules or for all experiments.

```
chi2_function(e; rule=1, systematics=true)
chi2_function!(e, name; rule=:all, errors, systematics=true)
```

The name of the χ^2 function of a rule, and changing it, with the errors of its nuisance parameters.

```
systematic_errors(e; rule=1, systematics=true)
systematic_errors!(e, errors; rule=:all, systematics=true)
```

The prior widths σ_k of the nuisance parameters.

```
chi2(test)                                      # all experiments and rules
chi2(test; experiment=nova, rule="Nu_E_Appearance")
chi2(test; systematics=false)                   # statistics only
chi2_function!(nova, "chiSpectrumCalib"; rule=2, errors=[0.025, 0.02, 0.05, 0.02])
systematic_errors!(nova, [0.05, 0.0001, 0.1, 0.0001]; rule=1)
```

6.4 User-defined systematics*

A χ^2 function that is not built in, for example for systematics that are correlated between rules or between a near and a far detector, is defined with

```
define_chi2(f, name, n_params; setup=current_setup())
```

Register `f(e::Experiment, rule::Int, x, errors) -> χ²` under `name`. `x` are the `n_params` nuisance parameters, which are minimised over, and `errors` their prior widths from the AEDL file or `chi2_function!`. `f` must include the priors.

Inside `f`, `true_rates(e; rule)` are the simulated rates n_i of a rule and `fit_rates(e; rule)` the signal s_i and background b_i predicted for the parameters being tested (before β_0 and systematics), `energy_window_bins(e; rule)` the bins to sum over, and `energy_scale_shift` and `poisson_chi2` help with the computation. The functions must be defined before AEDL files that use them are loaded. For example, a normalisation error common to the signal and background:

```
define_chi2("chiCommonNorm", 1) do e, rule, x, errors
    n = true_rates(e; rule)
    s, b = fit_rates(e; rule)
    β0 = e.data.rules[rule].bg_centers[1]
    sum(poisson_chi2(n[i], (1 + x[1]) * (s[i] + β0 * b[i])) for i in
     ↪  energy_window_bins(e; rule)) +
        (x[1] / errors[1])^2
end
nova = Experiment("NOvA.glb")
chi2_function!(nova, "chiCommonNorm"; errors=[0.05])
```

A rule whose events enter the χ^2 of another rule (a near detector, say) uses `chiZero`; the function of the other rule reads its rates with `true_rates` and `fit_rates`. For correlations between experiments, `experiments()` gives all experiments of the setup.

Chapter 7

Correlations, projections and degeneracies

7.1 Projections

The χ^2 of Chapter 6 tests one point of parameter space. To measure one parameter, say θ_{13}, all the others must be allowed to take the values that fit the data best, since they are not known exactly and can mimic a different θ_{13}. This is the *projection* of the χ^2 onto the θ_{13} axis (or *profiling*, or *marginalisation*):

$$\chi^2(\theta_{13}) = \min_{\theta_{12},\theta_{23},\delta_{\mathrm{CP}},\Delta m^2_{21},\Delta m^2_{31},\rho} \Big[\chi^2(\theta_{12},\theta_{13},\dots,\rho) + \chi^2_{\mathrm{prior}}\Big]. \tag{7.1}$$

More generally, any set of parameters can be kept fixed and the others minimised over. The χ^2 of a projection onto k parameters is distributed as χ^2 with k degrees of freedom; Table 7.1 gives the values for the usual confidence levels.

7.2 External information: priors

Parameters that an experiment does not measure well (like the solar parameters for a long-baseline experiment) or the matter density are constrained by external information, as Gaussian priors:

$$\chi^2_{\mathrm{prior}} = \sum_{k\ \mathrm{free}} \frac{(x_k - \hat{x}_k)^2}{\sigma_k^2} + \sum_{e\ \mathrm{free}} \frac{(\rho_e - \hat{\rho}_e)^2}{\sigma_\rho^2}, \tag{7.2}$$

summed over the fitted parameters. The centres $\hat{x}_k$ are the simulated (true) values unless given otherwise; parameters without σ_k have no prior. Any other prior is added as a function of the parameters.

7.3 Fitting

```
marginalize(start::OscParams; free, fixed, priors=(;), centers=simulated_params(),
            prior=nothing, experiment=:all, setup=current_setup()) -> FitResult
```

Minimise $\chi^2 + \chi^2_{\mathrm{prior}}$ over the parameters listed in `free` (or over all except those in `fixed`; by default over all), starting at `start`, with the fixed parameters at their values in `start`. Parameter names are those of Table 3.1 and `:ρ` for the matter densities of all experiments. `priors` are the σ_k as a named tuple, `centers` the $\hat{x}_k$, and `prior` a function `OscParams -> Float64` added to χ^2. The result has the fields `chi2`, `params` (the parameters at the minimum), `free` and `iterations` (the number of χ^2 evaluations).

	1σ (68.27%)	90%	2σ (95.45%)	99%	3σ (99.73%)
1 d.o.f.	1.00	2.71	4.00	6.63	9.00
2 d.o.f.	2.30	4.61	6.18	9.21	11.83
3 d.o.f.	3.53	6.25	8.02	11.34	14.16

Table 7.1: $\Delta\chi^2$ for confidence levels and 1–3 parameters (degrees of freedom).

```
test = with_param(truth, :sin²2013, 0.08)
r = marginalize(test; fixed=(:θ13,),
                priors=(θ12=0.03truth.θ12, Δm21=0.03truth.Δm21, ρ=0.05))
r.chi2, r.params.δ, r.params.ρ

# only δ and the density free, a non-Gaussian prior on θ23
marginalize(test; free=(:δ, :ρ), priors=(ρ=0.05,),
            prior = p -> p.sin²θ23 < 0.5 ? 0.0 : 4.0)
```

Note

The minimiser finds the minimum nearest to `start` (a local minimum). Start fits at the true parameters, or, to find degeneracies, at the points where they are expected (Section 7.6).

7.4 Scans

```
chi2_scan(base, x => xs; kwargs...) -> Vector
chi2_scan(base, x => xs, y => ys; kwargs...) -> Matrix
```

χ^2 on a grid of one or two parameters, with all others as in `base`. The parameter names include the derived ones of Table 3.1, such as `:sin²2013`. Without keyword arguments, each point is `chi2`; with any of the keywords of `marginalize`, each point is minimised over the other parameters, the scanned ones fixed. The matrix has one row per value of `x`.

Figure 7.1 shows the projection onto δ_{CP} for NOvA, Figure 7.2 a two-parameter scan for T2K and NOvA together:

```
δs = range(0, 2π, 73)
sys = chi2_scan(truth, :δ => δs)                                  # systematics only
fit = chi2_scan(truth, :δ => δs; free=(:θ23, :Δm31, :ρ), priors=(ρ=0.05,))
grid = chi2_scan(truth, :sin²2013 => range(0.06, 0.115, 56), :δ => δs)
```

7.5 The minimiser

GLoBES minimises with Powell's conjugate direction method and Brent's line minimisation [10], which needs no derivatives. Two strategies are available, chosen with `minimizer!`:

`:nested` (default) Powell's method over the free oscillation parameters. At every step, the systematics of each rule are minimised separately (Eq. (6.3)), each with its own Powell minimisation.

`:hybrid` One Powell minimisation over the free oscillation parameters and all nuisance parameters together, which can be faster with many rules.

The tolerances are 10^{-5} (relative) for oscillation parameters and 10^{-8} for systematics. Since Powell's method can stop early in narrow curved valleys, the minimisation over oscillation parameters is restarted with fresh directions until it no longer improves (at most 10 times).

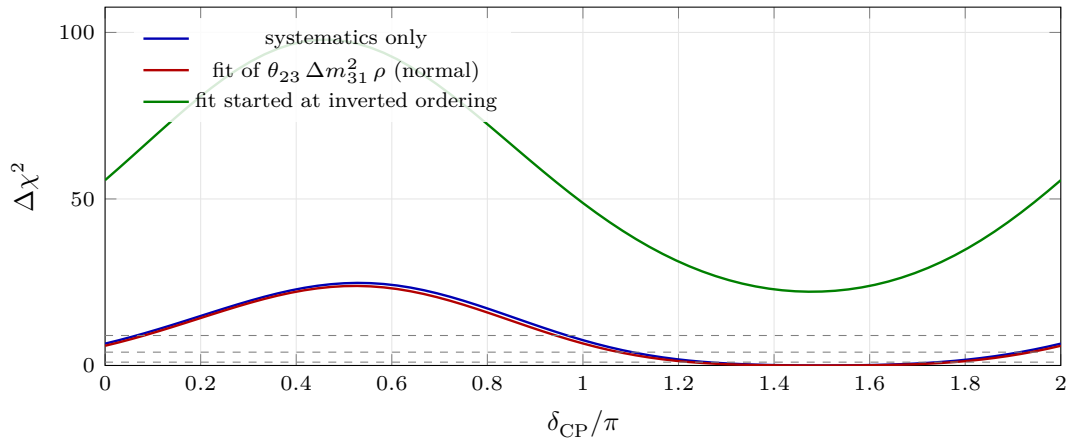


Figure 7.1: $\Delta\chi^2$ as a function of the tested δ_{CP} for `NOvA.glb`, true $\delta_{CP} = 3\pi/2$ and normal ordering: with all other parameters fixed at the true values, minimised over θ_{23}, Δm^2_{31} and ρ (5% prior), and the same fit started at $\Delta m^2_{31} \to -\Delta m^2_{31} + \Delta m^2_{21}$, which finds the best fit with inverted ordering (the sign degeneracy). Dashed lines: 1, 2 and 3σ for one parameter.

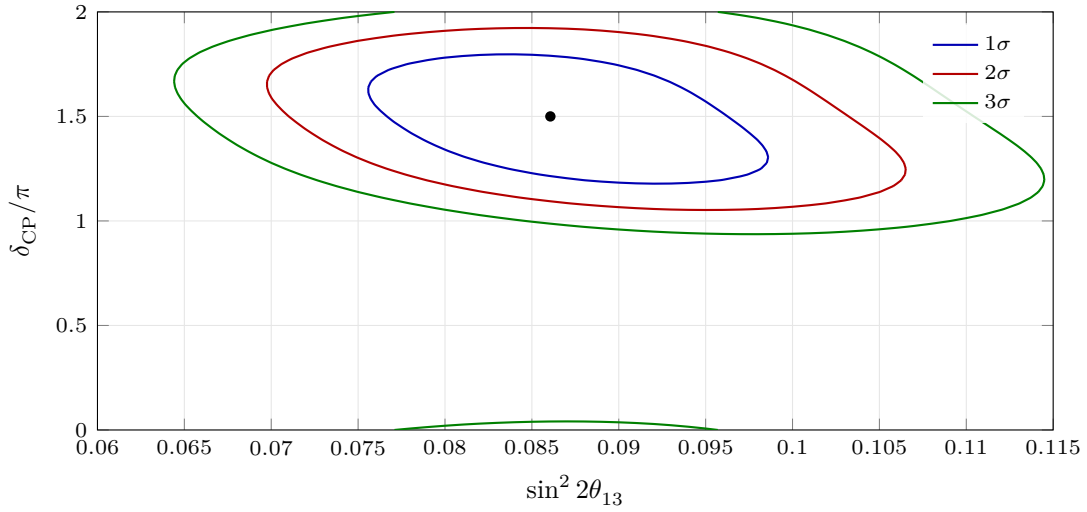


Figure 7.2: Allowed regions (2 degrees of freedom) in the $\sin^2 2\theta_{13}$–δ_{CP} plane for T2K and NOvA together, systematics only (`chi2_scan` with two parameters on a 56×73 grid). The dot marks the true values.

Difference from C GLoBES

C GLoBES writes Powell's test for replacing a direction with a macro `SQR(a)` that stores its argument in a static variable. Used twice in one expression this is undefined behaviour, and GCC computes a different formula than intended. C GLoBES also does not restart. Both can make fits stop at points that are not minima (Section 14.6).

7.6 Degeneracies

The appearance probability depends on θ_{13}, δ_{CP}, θ_{23} and the sign of Δm^2_{31} in similar ways, so that different parameter sets can give nearly the same event rates [9]:

- the *intrinsic* (θ_{13}, δ_{CP}) degeneracy;
- the *sign* degeneracy between normal and inverted ordering, ($\Delta m^2_{31} \to -\Delta m^2_{31} + \Delta m^2_{21}$);

- the *octant* degeneracy, $\theta_{23} \to \pi/2 - \theta_{23}$.

Each gives a local minimum of χ^2. Since the minimiser only finds the nearest minimum, a degenerate solution is found by starting the fit near it, as in Figure 7.1:

```
mirror = OscParams(truth; Δm31=-truth.Δm31 + truth.Δm21, θ23=π/2 - truth.θ23)
deg = marginalize(mirror; priors=(θ12=0.03truth.θ12, Δm21=0.03truth.Δm21, ρ=0.05))
deg.chi2, deg.params
```

The projection including degeneracies is the smallest χ^2 of all local minima. A coarse scan of χ^2 with systematics only (`chi2_scan`) is a good way to find where to start the fits.

Chapter 8

Changing experiments at runtime

Most properties of an experiment can be changed after it is loaded, for example to study the sensitivity as a function of the baseline or the running time. After any change, call `simulate!` again so that the simulated data match the new experiment.

8.1 Exposure

`target_mass(e)` `target_mass!(e, mass)`

The fiducial detector mass (in the units of the AEDL file, usually kt).

`running_time(e; flux=1)` `running_time!(e, years; flux=:all)`

The running time of one flux, or setting it for one or all fluxes.

`source_power(e; flux=1)` `source_power!(e, power; flux=:all)`

The source power: beam power, or useful parent decays per year.

Since the event numbers are proportional to all three, a scan over the exposure is

```
for years in (1, 2, 5, 10)
    running_time!(nova, years / 2)            # half neutrinos, half antineutrinos
    simulate!(truth)
    println(years, "  ", chi2(OscParams(truth; δ=0.5π)))
end
```

8.2 Baseline and matter profile

`baseline!(e, L)` changes the baseline and recomputes the density profile as the AEDL file defines it, `density_profile!(e, lengths, densities)` sets any profile (Section 4.3). Note that the flux decreases as $1/L^2$ when the baseline changes:

```
nf = Experiment("NFstandard.glb")
for L in 1000:1000:8000
    baseline!(nf, L)
    simulate!(truth)
    println(L, "  ", total_events(nf; rule=1))
end
```

8.3 Systematics and energy windows

`systematics!`, `chi2_function!` and `systematic_errors!` (Chapter 6) change the systematics treatment of rules, and `energy_window!` their energy window.

8.4 AEDL variables

AEDL files can use variables that are defined by the program. They are passed as keyword arguments when the experiment is loaded; numbers become variables and vectors become lists:

```
nf = Experiment("NFvar.glb"; BASELINE=3000, emax=50)
```

In the AEDL file, they are used like any variable, `$baseline = BASELINE` (Section 10.3). Lists are named with a leading `%` in AEDL; the keyword is given without it. This is the way to change things that cannot be changed after loading, such as the number of bins or the energy range.

8.5 The low-pass filter*

The filter of Section 4.4 is set in AEDL with `$filter_state` and `$filter_value`; at runtime, use `glbSetFilterStateInExperiment(i, GLB_ON)` and `glbSetFilterInExperiment(i, σ)` with the experiment number `i` (Chapter 13).

Chapter 9

Non-standard physics

GLoBES computes event rates from oscillation probabilities, and nothing else in the simulation depends on how these are computed. To simulate a model beyond three-flavour oscillations (sterile neutrinos, non-standard interactions, decoherence, decay, …) replace the probability engine.

```
probability_engine!(P!, extra; setup=current_setup())
standard_engine!(; setup=current_setup())
```

Use `P!` to compute probabilities, with the additional parameters named in `extra`, or go back to the standard engine. `P!(P, params, E, antineutrino, lengths, densities)` fills the 3×3 matrix `P[α, β]` $= P(\nu_\alpha \to \nu_\beta)$ for the `OscParams params` at energy `E` (GeV) through the layers of the matter profile (km and g/cm^3, already scaled by ρ).

After the engine is replaced, `OscParams` must have the additional parameters (`extra=(name=value, ...)`), and the data must be simulated again. The additional parameters work everywhere the standard ones do: they can be fitted, have priors, and be scanned.

As an example, here is an engine that multiplies the standard probabilities by $1 - \epsilon$, a crude model of neutrino decay or of a loss of normalisation:

```
engine = GLoBES.StandardEngine()
probability_engine!((:ϵ,)) do P, p, E, anti, lengths, densities
    GLoBES.set_oscillation_parameters!(engine, p.values[1:6])
    GLoBES.probability_matrix!(P, engine, anti ? -1 : +1, E, length(lengths),
                               lengths, densities, -1.0)
    P .*= 1 - p.ϵ
end

truth = OscParams(truth; extra=(ϵ=0.0,))
simulate!(truth)
r = marginalize(OscParams(truth; ϵ=0.1); free=(:ϵ, :023), priors=(023=0.05,))
r.params.ϵ                      # ≈ 0: the data have no loss
```

The standard engine used here (`GLoBES.StandardEngine`, `GLoBES.set_oscillation_parameters!` and `GLoBES.probability_matrix!`) is the one of Chapter 4: a model that modifies the Hamiltonian can build on its code in `src/probability.jl`.

Note

The engine is called for every sampling point of every experiment for every χ^2 evaluation, so it should be fast and should not allocate memory.

Part II

The Abstract Experiment Definition Language

Chapter 10

Getting started with AEDL

10.1 The concept

An experiment in GLoBES has one neutrino source (a point source, stationary in time), one baseline with its matter density profile, and one detector. The detector is described by its response to neutrinos of each flavour: cross sections, energy resolution, efficiencies and backgrounds. The building blocks are

fluxes (`nuflux`) of the source, from a table or built in;

cross sections (`cross`), from tables;

energy resolutions (`energy`), a resolution function or a matrix;

channels (`channel`): one flux, one initial and one final flavour, one cross section and one energy resolution, with efficiencies;

rules (`rule`): signal and background, each a weighted sum of channels, with a χ^2 function for systematics.

The χ^2 of an experiment is the sum of the χ^2 of its rules, and the χ^2 of a setup the sum over its experiments (Figure 10.1). Chapter 5 gives the formulas.

10.2 A simple example

The file `test/data/Simple.glb` describes a superbeam with a water Cherenkov detector at 295 km:

```
%!GLoBES

$version = "3.0.0"

/* A simple superbeam: the example of the AEDL chapter of the GLoBES.jl manual.
   A 0.77 MW beam, a 22.5 kt water Cherenkov detector at 295 km. */

/* Beam */
nuflux(#plus)<
  @flux_file = "JHFplus.dat"
  @time      = 5        /* years */
  @power     = 0.77     /* MW */
  @norm      = 6.93185
>

/* Detector */
```

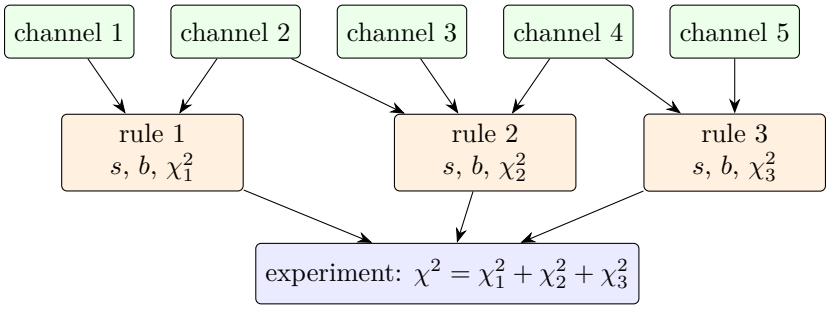


Figure 10.1: Channels, rules and experiments. A channel can be used in several rules, as signal in one and as background in another.

```
$target_mass = 22.5     /* kt */

/* Energy binning and sampling */
$emin = 0.4
$emax = 1.2
$bins = 16
$sampling_min    = 0.1
$sampling_max    = 1.5
$sampling_points = 56

/* Baseline: average density along 295 km */
$profiletype = 1
$baseline    = 295.0

/* Cross sections */
cross(#CC)< @cross_file = "XCC.dat" >
cross(#NC)< @cross_file = "XNC.dat" >

/* Energy resolution: sigma(E) = 0.085 GeV */
energy(#res)<
  @type    = 1
  @sigma_e = {0, 0, 0.085}
>

/* Channels */
channel(#mu_mu)<     @channel = #plus: +: m: m: #CC: #res >
channel(#e_e)<       @channel = #plus: +: e: e: #CC: #res >
channel(#nc_bg)<     @channel = #plus: +: NOSC_m: NOSC_m: #NC: #res >

/* An energy dependent efficiency for the appearance signal */
%centers = bincenter()
%eff = interpolation({0.4, 0.6, 1.2}, {0.3, 0.5, 0.5}, 1, %centers)

channel(#mu_e_eff)<
  @channel = #plus: +: m: e: #CC: #res
  @post_smearing_efficiencies = copy(%eff)
>

/* Rules */
rule(#appearance)<
  @signal          = 1.0 @ #mu_e_eff
  @background      = 0.005 @ #nc_bg : 0.5 @ #e_e : 0.001 @ #mu_mu
  @signalerror     = 0.05 : 0.0001
  @backgrounderror = 0.05 : 0.0001
  @sys_on_function  = "chiSpectrumTilt"
```

```
62    @sys_off_function = "chiNoSysSpectrum"
63  >
64
65  rule(#disappearance)<
66    @signal          = 0.9 @ #mu_mu
67    @background      = 0.005 @ #nc_bg
68    @signalerror     = 0.025 : 0.0001
69    @backgrounderror = 0.2 : 0.0001
70    @sys_on_function  = "chiSpectrumTilt"
71    @sys_off_function = "chiNoSysSpectrum"
72    @energy_window    = 0.5 : 1.2
73  >
```

It defines a flux from a table (lines 9–14), the detector mass, the energy bins and sampling points (lines 17–26), the baseline and matter profile (lines 29–30), two cross sections, a Gaussian energy resolution, four channels, and two rules. The channel `#mu_e_eff` has an energy dependent efficiency computed in AEDL by interpolation (lines 47–53). Loaded with

```
simple = Experiment("Simple.glb")
```

it can be used like any other experiment.

10.3 Syntax

File structure. An AEDL file starts with the eight characters `%!GLoBES`. It should declare the GLoBES version it is written for, `$version = "3.0.0"`. Comments are as in C, `/* ... */` and `// ...` to the end of the line.

Global and local variables. Variables starting with `$` are global properties of the experiment (`$baseline = 295`). Variables starting with `@` belong to an environment (`@time = 5` inside `nuflux`). Numbers are converted to integers where integers are expected. Lists are written in braces, `$densitytab = {2.8, 3.5}`.

Environments and names. Fluxes, cross sections, energy resolutions, channels and rules are environments,

```
channel(#name)<
   @channel = ...
>
```

named with a `#`. Names are numbered in the order of their first appearance within each type (starting at 1 in GLoBES.jl), and must be unique across all types. A name can be declared before it is defined with an empty environment, `channel(#later)<>`. `rule_names` and `channel_names` return the names of an experiment.

User variables and expressions. Variables without prefix are constants for use in expressions:

```
L = 1300
$baseline = L
$emax = 2 * sqrt(L / 100) + 0.5^2
```

The operators are `+ - * / ^`, and the functions `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `log`, `log10`, `exp` and `sqrt`, which apply to every element of a list. Variables can also be defined by the program: `Experiment(file; L=810)` (Chapter 8). Lists are stored in variables starting with `%`, `%effs = {0.2, 0.4, 0.6}`.

List functions.

`bincenter()` the centres of the energy bins defined so far;

`samplingbincenter()`
the centres of the sampling points;

`interpolation(x, y, k, z)`
the values at `z` of the linear ($k = 1$) or natural cubic spline ($k = 2$) interpolation of the points (x, y);

`copy(l)` a copy of a list, needed when a list is used twice;

`reverse(l)` the list in reverse order;

`echo(x), echon(x), line(n)`
print values (for debugging AEDL files), without or with line feeds, or n line feeds.

Includes and several experiments. `include "file.glb"` inserts another file (which need not start with `%!GLoBES`); includes can be nested ten levels deep. Several experiments are separated by `#NEXT#`, which resets all names, so that

```
%!GLoBES
include "NOvA.glb"
#NEXT#
include "T2K.glb"
```

defines two experiments, loaded with `load_experiments`.

Where files are found. Included files, flux files and cross section files are looked up in the current directory, in the directory of the AEDL file that names them, and in the directories of `GLB_PATH`.

Difference from C GLoBES

C GLoBES looks for data files only in the current directory and in `GLB_PATH`, not next to the AEDL file, so that loading an experiment from another directory fails to find its flux and cross section files.

Chapter 11

Experiment definition with AEDL

This chapter lists all AEDL directives. Defaults are given in brackets.

11.1 Source and integrated luminosity

`nuflux(#name)< ... >` (or the older `flux`, see below)	
`@flux_file = "file"`	flux table (see below)
`@builtin = n`	built-in flux: 1 μ^+ decay, 2 μ^- decay (neutrino factory), 3 β^+ decay, 4 β^- decay (beta beam)
`@time`	running time [1]
`@power`	source power [1], e.g. MW
`@stored_muons`, `@stored_ions`	useful parent decays per year [1]
`@norm`	normalisation of the flux table [1]
`@parent_energy`	muon energy in GeV (built-in 1, 2)
`@gamma`, `@end_point`	Lorentz factor of the ions and endpoint of the decay in GeV (built-in 3, 4)
`$target_mass`	fiducial detector mass, e.g. kt

Flux files have 501 lines of seven columns, with equidistant energies:

$$E\,[\mathrm{GeV}] \quad \phi_{\nu_e} \quad \phi_{\nu_\mu} \quad \phi_{\nu_\tau} \quad \phi_{\bar\nu_e} \quad \phi_{\bar\nu_\mu} \quad \phi_{\bar\nu_\tau}$$

Lines starting with `#` are comments. The flux is interpolated linearly and is zero outside the table. Unused flavours must be given as zeros.

Normalisation. The flux in Eq. (5.2) is $\mathcal{N}/L^2 \times$`@norm`$\times$`@power`$\times$`@stored_muons`$\times$`@time`$\times\phi(E)$, with $\mathcal{N} = 1$ for `nuflux` tables. Together with the cross sections in $10^{-38}\,\mathrm{cm}^2$ per nucleon and the target mass M, the number of events per GeV is

$$\frac{dN}{dE} = \frac{\texttt{@norm} \times \texttt{@power} \times \texttt{@time} \times M}{(L/\mathrm{km})^2}\,\phi(E)\,\frac{\sigma(E)}{10^{-38}\,\mathrm{cm}^2}\,P(E). \tag{11.1}$$

If the table gives the number of neutrinos per GeV and per area A at a distance L_0 for an integrated luminosity $\mathcal{L}$ (in the units $\mathcal{L}_u$ chosen for `@power`$\times$`@time`), the normalisation is

$$\texttt{@norm} = \frac{\mathrm{cm}^2}{A}\left(\frac{L_0}{\mathrm{km}}\right)^2 \frac{N_\mathrm{target}}{M_u} \times 10^{-38} \times \frac{\mathcal{L}_u}{\mathcal{L}}, \tag{11.2}$$

with N_target/M_u the number of target nucleons per unit of the target mass (6.022×10^{32} per kt). The older environment `flux` multiplies by $\mathcal{N} = 5.1989$ in addition; replace `flux` by `nuflux` and multiply `@norm` by 5.1989 to convert. Built-in fluxes have their own normalisation and need no `@norm`.

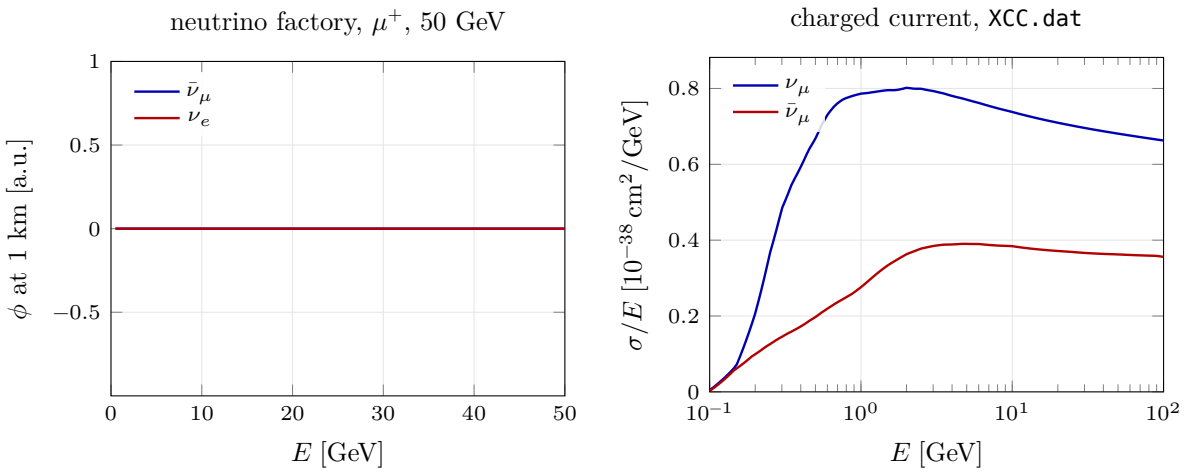


Figure 11.1: Left: the built-in neutrino factory flux (`@builtin = 1`) of `NFstandard.glb`, `flux(nf, E; flavour=:μ, antineutrino=true, L=1)`. Right: the charged-current cross sections per nucleon divided by energy, `cross_section(nf, E; flavour=:μ) / E`.

Built-in fluxes. The neutrino factory flux (`@builtin = 1, 2`) is that of muons of energy E_μ (`@parent_energy`) decaying at rest in the beam direction, with $x = 2E/(E_\mu(1+\beta))$,

$$\phi_{\bar{\nu}_\mu}\ (\phi_{\nu_\mu}) \propto \frac{\gamma(1+\beta)}{\pi m_\mu}\, x^2(3-2x), \qquad \phi_{\nu_e}\ (\phi_{\bar{\nu}_e}) \propto \frac{\gamma(1+\beta)}{\pi m_\mu}\, 6x^2(1-x), \tag{11.3}$$

for μ^+ (μ^-) decays, up to the endpoint $x = 1$. The beta beam flux (`@builtin = 3, 4`) is the spectrum of nuclear beta decay with endpoint E_0 (`@end_point`), boosted by γ (Figure 11.1):

$$\phi(E) = \frac{\gamma^2}{\pi\, g(y_e)\, E}\, x^3(1-x)\sqrt{(1-x)^2 - y_e^2}, \qquad x = \frac{E}{2\gamma(E_0+m_e)}, \quad y_e = \frac{m_e}{E_0+m_e}, \tag{11.4}$$

with the phase space integral $g(y) = \frac{1}{60}\big[\sqrt{1-y^2}(2-9y^2-8y^4) + 15y^4 \ln\frac{1+\sqrt{1-y^2}}{y}\big]$ (normalised to a distance of 1 km).

Difference from C GLoBES

C GLoBES uses $\pi = 3.1415$, $\log_{10}$ instead of ln in $g(y)$, an old value of Avogadro's number, and rounded muon and electron masses. Its flux interpolation divides the energy range of the table into 501 instead of 500 steps, which evaluates every flux at 1.002 times the requested energy, and the built-in fluxes are tabulated only up to 500/501 of their endpoint (Section 14.2).

11.2 Baseline and matter density profile

`$baseline`	baseline in km
`$profiletype = 1`	a single layer with the average density along `$baseline`
`$profiletype = 2`	`$densitysteps` layers of equal length with the Earth's density
`$profiletype = 3`	layers `$lengthtab = {...}` (km) with densities `$densitytab = {...}` (g/cm^3); the baseline is their total length

A constant density ρ is `$profiletype = 3`, `$densitytab = {ρ}`, `$lengthtab = {L}`. See Section 4.3.

11.3 Cross sections

`cross(#name)< @cross_file = "file" >` reads a table of 1001 lines with equidistant $\log_{10}(E/\mathrm{GeV})$:

$$\log_{10} E \quad \hat\sigma_{\nu_e} \quad \hat\sigma_{\nu_\mu} \quad \hat\sigma_{\nu_\tau} \quad \hat\sigma_{\bar\nu_e} \quad \hat\sigma_{\bar\nu_\mu} \quad \hat\sigma_{\bar\nu_\tau}, \qquad \hat\sigma = \frac{\sigma(E)}{E}\ [10^{-38}\,\mathrm{cm}^2/\mathrm{GeV}].$$

The table is interpolated linearly in $\log_{10} E$ and is zero outside. The files shipped with GLoBES, `XCC.dat` (charged current), `XNC.dat` (neutral current) and `XQE.dat` (quasi-elastic), cover 0.1 to 1000 GeV and are per nucleon of an isoscalar target.

11.4 Channels

```
channel(#name)<
   @channel = #flux : +|- : initial : final : #cross : #energy
   @pre_smearing_efficiencies  = {...}    // one per sampling point [1]
   @pre_smearing_background    = {...}    // one per sampling point [0]
   @post_smearing_efficiencies = {...}    // one per bin [1]
   @post_smearing_background   = {...}    // one per bin [0]
>
```

`+` and `-` are neutrinos and antineutrinos of the flux. Flavours are `e`, `m`, `t` (also `electron`, `muon`, `tau`); `NOSC_e` etc. give a channel without oscillations ($P = 1$ for equal flavours), for neutral currents and intrinsic backgrounds. The efficiencies and backgrounds are ϵ^{pre}, B^{pre}, ϵ^{post} and B^{post} of Eqs. (5.1) and (5.3).

11.5 Energy resolution

Binning.

`$emin`, `$emax`	range of reconstructed energies in GeV
`$bins`	number of bins of equal width
`$binsize = {...}`	bin widths instead, for unequal bins (they must add up to the range)
`$sampling_min`, `$sampling_max`	range of true energies [`$emin`, `$emax`]
`$sampling_points`	number of sampling points [`$bins`]
`$sampling_stepsize = {...}`	widths of the sampling steps instead

Resolution functions.

```
energy(#name)<
   @type = 1                        // 2: reduce the width by the filter
   @sigma_function = #standard      // or #inverse_beta
   @sigma_e = {0.15, 0, 0}          // σ(E) = 0.15 E
>
```

The smearing matrix is computed from Eq. (5.5) with the resolution functions of Section 5.3.

Smearing matrices. The matrix K_{ij} can also be given row by row, one row per bin i, each with the first and last sampling point (counted from 0) of its non-zero entries followed by the entries:

```
energy(#migration)<
   @energy = {0, 2, 0.6, 0.3, 0.1} :
             {0, 3, 0.2, 0.5, 0.2, 0.1} :
             ...
             {17, 19, 0.1, 0.3, 0.6} ;
>
```

Matrices from detector simulations (for example the beta beam files `*EffMig350.dat`) are included this way.

Filter. `$filter_state = 1` and `$filter_value =` σ_f (GeV) switch on the low-pass filter of Section 4.4.

11.6 Rules and systematics

```
rule(#name)<
   @signal          = 0.5 @ #channel_1
   @background      = 0.001 @ #channel_2 : 0.005 @ #channel_3
   @signalerror     = 0.05 : 0.0001        // σ1 : σ2
   @backgrounderror = 0.1 : 0.0001         // σ3 : σ4
   @backgroundcenter = 1.0 : 0.0           // β0 : τ0 (better not used)
   @sys_on_function  = "chiSpectrumTilt"
   @sys_off_function = "chiNoSysSpectrum"
   @energy_window    = 0.5 : 3.0            // GeV
>
```

`@signal` and `@background` are the coefficients a_k and b_k of Eq. (5.4). The errors are those of the built-in χ^2 functions (Section 6.2.1). Functions with other numbers of parameters (user-defined ones) take their errors as lists, `@sys_on_errors = {0.02, 0.05}` and `@sys_off_errors`, which also override `@signalerror` and `@backgrounderror` for built-in functions. The energy window restricts the bins used in the χ^2 of the rule [`$emin` : `$emax`]; with an energy calibration error, it should leave a margin of a few times the error at both ends. GLoBES 2 files may use `@errordim_sys_on` and `@errordim_sys_off` (Table 6.1).

11.7 Testing AEDL files

Load the file and look at what it defines:

```
e = Experiment("MyExperiment.glb")       # parse errors are thrown with file and line
display(e)                               # baseline, mass, bins, rules
channel_names(e), rule_names(e)
simulate!(truth)
event_rates(e; rule=1)                   # signal and background per bin
[sum(channel_rates(e, c)) for c in eachindex(channel_names(e))]
```

Compare the total event numbers with those published for the experiment; most problems are a wrong flux normalisation (`@norm`) or efficiencies.

Part III

Reference

Chapter 12

Function reference

All functions of the high-level API, grouped by topic. Every function also has a docstring, shown with `?name` in the Julia REPL. Keyword arguments `setup` default to `current_setup()`; functions taking an experiment use the experiment's setup.

Setups (Section 3.6)

`Setup()`	a new, empty setup
`current_setup()`	the setup used by default
`with_setup(f, setup)`	call `f()` with `setup` as the current setup

Oscillation parameters (Section 3.2)

`OscParams(; θ12, θ13, θ23, δ=0, Δm21, Δm31, ρ=1, extra=())`	create parameters
`OscParams(p; changes...)`	copy with changes
`with_param(p, name, value)`	copy with one change; also `:sin²2θ13` etc.
`p.θ13, p.sin²2θ13, p.ρ, ...`	read parameters

Experiments (Chapters 3, 8)

`Experiment(file; setup, variables...)`	load an AEDL file with one experiment
`load_experiments(file; setup, variables...)`	load all experiments of a file
`experiments(; setup)`	the experiments of a setup
`clear_experiments!(; setup)`	remove all experiments
`name(e)`	name of the AEDL file
`rule_names(e), channel_names(e)`	names from the AEDL file
`baseline(e), baseline!(e, L)`	baseline in km
`density_profile(e), density_profile!(e, l, ρ)`	matter density layers
`prem_profile(L, n), average_profile(L)`	the Earth's density along a baseline
`target_mass(e), target_mass!(e, m)`	detector mass
`running_time(e; flux), running_time!(e, t; flux)`	running time
`source_power(e; flux), source_power!(e, p; flux)`	source power
`energy_range(e), bin_centers(e), bin_widths(e)`	reconstructed energy bins
`energy_window(e; rule), energy_window!(e, lo, hi; rule)`	energy window of a rule
`energy_window_bins(e; rule)`	bins inside the energy window
`flux(e, E; flux, flavour, antineutrino, L)`	source flux
`cross_section(e, E; xsec, flavour, antineutrino)`	cross section in $10^{-38}\,\mathrm{cm}^2$

Probabilities (Chapter 4)

`probability(p, from, to; E, L, ρ=0, antineutrino=false)`	constant density or vacuum
`probability(p, from, to, e; E, antineutrino=false)`	along an experiment's profile

Simulated data and event rates (Chapter 5)

`simulate!(truth; setup)`	compute the simulated data
`simulated_params(; setup)`	parameters of the simulated data
`event_rates(e; rule)`	simulated rates of a rule per bin
`event_rates(e, p; rule)`	predicted rates for parameters `p`
`total_events(e; rule)`, `total_events(e, p; rule)`	sum over the energy window
`channel_rates(e, channel; smeared=true)`	simulated rates of a channel

χ^2 and systematics (Chapter 6)

`chi2(p; experiment, rule, systematics=true, setup)`	χ^2 minimised over systematics
`systematics!(e, on; rule)`, `systematics!(on; setup)`	switch systematics on or off
`chi2_function(e; rule, systematics)`	name of a rule's χ^2 function
`chi2_function!(e, name; rule, errors, systematics)`	change it
`systematic_errors(e; rule, systematics)`	widths of the nuisance priors
`systematic_errors!(e, errors; rule, systematics)`	change them
`define_chi2(f, name, n; setup)`	define a χ^2 function
`true_rates(e; rule)`, `fit_rates(e; rule)`	rates inside a χ^2 function
`poisson_chi2(n, N)`	Poisson χ^2 of one bin
`energy_scale_shift(e, rates, b)`	apply an energy calibration error
`energy_scale_shift(rates, b; centers, widths)`	the same for any binning

Fits (Chapter 7)

`marginalize(p; free, fixed, priors, centers, prior, experiment, setup)`	minimise over parameters; returns a `FitResult`
`r.chi2, r.params, r.free, r.iterations`	fields of a `FitResult`
`chi2_scan(p, x => xs; kwargs...)`	one-parameter scan
`chi2_scan(p, x => xs, y => ys; kwargs...)`	two-parameter scan
`minimizer!(:nested | :hybrid; setup)`	choose the minimisation strategy

Non-standard physics (Chapter 9)

`probability_engine!(P!, extra; setup)`	replace the probability engine
`standard_engine!(; setup)`	restore the standard engine

Errors

`GLoBESError`	thrown by the high-level API
`AEDLError`	an error in an AEDL file

Accessing the engine

The data of an experiment are in `e.data`, an `ExperimentData` structure with the fields of the C structure `glb_exp` (1-based): `e.data.rules[r].bg_centers`, `e.data.smear_data[1].bincenter`, `e.data.lengthtab` and so on. They are useful in user-defined χ^2 functions, but changing them directly bypasses the consistency checks of the functions above.

Chapter 13

The `glb*` compatibility layer

The functions with the names of the C API make it possible to translate C GLoBES programs line by line; `examples/example1.jl` to `example6.jl` and `example-tour.jl` are such translations of the C examples. They are documented in the C GLoBES manual [3]; this chapter lists how they differ from C and what they correspond to in the high-level API.

13.1 Conventions

- **Numbering starts at 1.** Experiments, rules, channels, fluxes and cross sections are numbered from 1, and the parameter indices are `GLB_THETA_12` = 1, `GLB_THETA_13` = 2, `GLB_THETA_23` = 3, `GLB_DELTA_CP` = 4, `GLB_DM_21` = 5, `GLB_DM_31` = 6. `GLB_ALL` selects all experiments or rules.

- **Parameter vectors.** `glbAllocParams()` returns a `GlbParams` with fields `osc`, `density` and `iterations`; undefined entries are `NaN`. `glbFreeParams` and `glbFreeProjection` do nothing. Since the vectors depend on the number of experiments, allocate them after loading the experiments.

- **Pointers** (`glbGetRuleRatePtr` etc.) return the engine's vectors, which change when rates are recomputed. Copy them to keep values.

- **Output.** Printing functions take an optional `IO` as first argument (`glbPrintParams(io, p)`).

- **Errors** are printed (`glbSetVerbosityLevel`) and signalled by return values, as in C. Fatal errors throw a `GLoBESError`.

- χ^2 **functions** for `glbDefineChiFunction` have the signature `f(exp, rule, n_params, x, errors, user_data)` with 1-based `exp` and `rule`.

- **Probability engines** for `glbRegisterProbabilityEngine` are `prob(P, cp_sign, E, psteps, lengths, densities, filter_sigma, user_data)`, `set(p::GlbParams, user_data)` and `get(p, user_data)`; they return 0 on success.

- **State.** The functions act on the current setup (Section 3.6).

13.2 Correspondence with the high-level API

C / glb*	high-level API
glbInit	(automatic)
glbInitExperiment(file)	Experiment(file), load_experiments(file)
glbDefineAEDLVariable, glbDefineAEDLList	keyword arguments of Experiment
glbClearExperimentList	clear_experiments!()
glbNumOfExps	length(experiments())
glbDefineParams, glbSetOscParams, glbSetDensityParams	OscParams(...), OscParams(p; ...)
glbSetOscillationParameters + glbSetRates	simulate!(truth)
glbChiSys(p, exp, rule)	chi2(p; experiment, rule)
glbChiNP, glbChiTheta13, …, glbChiAll	marginalize(p; free=..., fixed=...)
glbChiTheta13Delta	marginalize(p; fixed=(:θ13, :δ))
glbSetProjection, glbDefineProjection	free, fixed keywords
glbSetCentralValues, glbSetInputErrors	centers, priors keywords
glbRegisterPriorFunction	prior keyword
glbSelectMinimizer	minimizer!
glbSwitchSystematics	systematics!
glbSetChiFunction, glbGetChiFunction	chi2_function!, chi2_function
glbSetSysErrorsList, glbGetSysErrorsListPtr	systematic_errors!, systematic_errors
glbSetSignalErrors, glbSetBGErrors	systematic_errors!
glbDefineChiFunction	define_chi2
glbGetRuleRatePtr, glbGetSignalRatePtr, glbGetBGRatePtr	event_rates(e; rule)
glbGetSignalFitRatePtr, glbGetBGFitRatePtr	fit_rates(e; rule), event_rates(e, p; rule)
glbGetChannelRatePtr, glbGetChannelRates	channel_rates
glbTotalRuleRate	total_events
glbShiftEnergyScale	energy_scale_shift
glb_likelihood, glb_prior	poisson_chi2, (write $((x-x_0)/\sigma)^2$)
glbVacuumProbability, glbConstantDensityProbability	probability(p, a, b; E, L, ρ)
glbProfileProbability	probability(p, a, b, e; E)
glbStaceyProfile, glbAverageDensityProfile	prem_profile, average_profile
glbSetProfileDataInExperiment, glbGetProfileDataInExperiment	density_profile!, density_profile
glbSetBaselineInExperiment	baseline!
glbSetTargetMass, glbSetRunningTime, glbSetSourcePower	target_mass!, running_time!, source_power!
glbSetEnergyWindow, glbGetEnergyWindowBins	energy_window!, energy_window_bins
glbGetBinCentersListPtr, glbGetBinSizeListPtr	bin_centers, bin_widths
glbGetEminEmax	energy_range
glbNameToValue, glbValueToName	rule_names, channel_names; names are accepted for rule
glbFlux, glbXSection	flux, cross_section
glbRegisterProbabilityEngine	probability_engine!, standard_engine!
glbSetVerbosityLevel	(exceptions)

13.3 A translated program

The C code of a $\sin^2 2\theta_{13}$–δ_{CP} scan (the first example of the C manual) and its GLoBES.jl versions:

```
# glb* layer: as in C
true_values = glbAllocParams(); test_values = glbAllocParams()
```

```
glbDefineParams(true_values, θ12, θ13, θ23, δ, Δm21, Δm31)
glbSetDensityParams(true_values, 1.0, GLB_ALL)
glbCopyParams(true_values, test_values)
glbSetOscillationParameters(true_values); glbSetRates()
for x in -4:0.04:-2, y in 0:4:200
    glbSetOscParams(test_values, asin(sqrt(10^x)) / 2, GLB_THETA_13)
    glbSetOscParams(test_values, y * π / 180, GLB_DELTA_CP)
    res = glbChiSys(test_values, GLB_ALL, GLB_ALL)
end

# high-level API
truth = OscParams(; θ12, θ13, θ23, δ, Δm21, Δm31)
simulate!(truth)
grid = chi2_scan(truth, :sin²2θ13 => exp10.(-4:0.04:-2), :δ => deg2rad.(0:4:200))
```

Chapter 14

Differences from C GLoBES

GLoBES.jl was developed as a port of C GLoBES 3.0.11 and is tested against it. Where the C library has a bug, GLoBES.jl does the right thing instead. This chapter lists every such difference, how it shows, and how much it changes results. The test suite compares GLoBES.jl with a C GLoBES built with the patch `test/reference/c-fixes.patch`, which applies the same fixes to the C code where that is a small change (Sections 14.1, 14.2 and 14.3). With it, both agree to rounding errors except where noted.

14.1 Physical constants

	C GLoBES	GLoBES.jl (CODATA 2018)
$\hbar c$	$1.974\,7235 \times 10^{-10}$ eV km	$1.973\,269\,804 \times 10^{-10}$ eV km
$\sqrt{2} G_F N_A$	7.5×10^{-14} eV per g/cm^3	7.6325×10^{-14} eV per g/cm^3
π (in fluxes)	3.1415	π
N_A (flux normalisation)	6.02204×10^{23}	$6.022\,140\,76 \times 10^{23}$
m_μ, m_e	0.1056, 0.511×10^{-3} GeV	0.105 658 3755, $0.510\,998\,95 \times 10^{-3}$ GeV

The first shifts every oscillation phase by 0.07% (equivalent to a change of Δm^2 by that amount), the second makes the matter potential 1.7% larger, which is well within the usual density uncertainty but noticeable in precise comparisons. The others change built-in fluxes by 10^{-5} to 10^{-4}.

14.2 Flux tables

A flux table has 501 equidistant energies $E_0, \ldots, E_{500}$. To interpolate, C GLoBES computes the step as $(E_{500} - E_0)/501$ instead of /500, so it reads the table at the wrong position: every flux is evaluated at 1.002 times the requested energy (more precisely at $E_0 + \frac{501}{500}(E - E_0)$), and the last 0.2% of the table is never used. For the built-in fluxes, which are tabulated at initialisation, it also places the 501 points up to 500/501 of the parent energy, so that the highest-energy part of the neutrino factory spectrum, where the ν_μ flux is largest, is cut off. GLoBES.jl uses the correct step and tabulates the built-in fluxes up to the kinematic endpoint. The beta beam flux uses the phase space integral $g(y_e)$ with $\log_{10}$ instead of ln, which changes its normalisation by about 0.35%.

Together with the constants above, this changes total event numbers by 0.1–1% for the superbeam, reactor and beta beam experiments of `test/data`, and by up to 3.5% for the neutrino factory appearance channels near the end of the spectrum.

14.3 Background tilt prior

In `chiSpectrumTilt` and `chiTotalRatesTilt`, the nuisance parameter of the background tilt ζ_4 enters the rates as $\zeta_4/\Delta E$, like the signal tilt. C GLoBES applies the prior to $\zeta_4/\Delta E$, while the prior of the signal tilt is on ζ_2:

$$\text{C:}\quad \frac{\zeta_2^2}{\sigma_2^2} + \frac{(\zeta_4/\Delta E - \tau_0)^2}{\sigma_4^2}, \qquad \text{GLoBES.jl:}\quad \frac{\zeta_2^2}{\sigma_2^2} + \frac{(\zeta_4 - \tau_0)^2}{\sigma_4^2}.$$

In C the background tilt error is therefore $\sigma_4\,\Delta E$ instead of the σ_4 written in the AEDL file, a factor of 0.8 for T2K and 46 for the neutrino factory. (In `chiSpectrumCalib` both priors are on the parameters.)

14.4 Empty bins in the Poisson χ^2

C GLoBES returns 10^{100} from `glb_likelihood` whenever the prediction is $N \le 0$, even for a bin with no data, $n = 0$. Experiments with empty bins (for example a beta beam or neutrino factory with bins above the beam energy, `NF_hR_lT.glb` and `BBvar_WC.glb` in `test/data`) then have $\chi^2 = 10^{100}$ for every hypothesis, which is meaningless; and when systematics push a prediction through zero, χ^2 jumps from a finite value to 10^{100}, which disturbs the minimisers. GLoBES.jl gives empty bins $\chi^2_{\mathrm{P}} = 2|N|$ and continues the Poisson χ^2 smoothly below $N = 10^{-10}$ (Section 6.1).

14.5 Energy scale shift

C GLoBES applies an energy calibration error b (in `chiSpectrumCalib` and `glbShiftEnergyScale`) by linear interpolation of the bin contents at the shifted positions $\delta_i = b(i + t_0 + \frac{1}{2}) + i$: $\mathcal{S}_b[s]_i = (1+b)\,[s_k + (s_{k+1} - s_k)(\delta_i - k)]$, $k = \lfloor \delta_i \rfloor$. This has three problems:

1. The interpolation is between bin *contents*. Since δ_i moves across bin boundaries at $b = 0$, the result has a kink at $b = 0$ in every bin: $\chi^2(b)$ has a cusp at its minimum (Figure 6.2), and its curvature, which determines the effect of the error, is ill-defined.
2. For $k = N_{\mathrm{bins}} - 1$ (shifts beyond the upper end), it uses $s_k(\delta_i - k) + s_k$, which counts the last bin twice instead of assuming zero events above the range.
3. It assumes bins of equal width; with `$binsize` the shift is wrong.

GLoBES.jl integrates the monotone interpolation of the cumulative spectrum between the scaled bin edges (Section 6.2.2), which avoids all three. The test suite therefore uses `chiSpectrumTilt` instead of `chiSpectrumCalib` in its comparisons with C, and tests the new shift against exact integrals of known spectra.

14.6 The minimiser

Powell's method replaces the direction of largest decrease by the new direction if [10]

$$t = 2(f_0 - 2f_N + f_E)\,(f_0 - f_N - \Delta f)^2 - \Delta f\,(f_0 - f_E)^2 < 0. \tag{14.1}$$

C GLoBES writes this with the Numerical Recipes macro `SQR(a)`, defined as `(sqrarg=(a), sqrarg*sqrarg)` with a single static variable `sqrarg`. Two `SQR` in one expression are undefined behaviour; GCC evaluates both assignments before both products and computes $t = (2(f_0 - 2f_N + f_E) - \Delta f)(f_0 - f_E)^2$. The test then accepts or rejects new directions for the wrong reasons. In addition, a single Powell run can stop early in a long curved valley, as around

degenerate solutions. GLoBES.jl uses the formula above and restarts the minimisation from the result with fresh directions until a whole run no longer improves χ^2 (at most ten times). Fits may therefore end at a lower χ^2 than in C GLoBES.

14.7 Data file lookup

C GLoBES looks for flux and cross section files only in the current directory and in `GLB_PATH`. GLoBES.jl also looks next to the AEDL file, so that `Experiment("/path/to/exp.glb")` works from any directory.

14.8 State and interface

- All state belongs to a `Setup` instead of global variables (Section 3.6).
- The high-level API raises exceptions instead of printing errors and returning codes; the `glb*` functions keep the C behaviour.
- Numbering starts at 1 (Chapter 13).
- The rates of a rule outside its energy window are zero (C leaves them uninitialised).

Appendix A

Experiment files

The AEDL files of the C GLoBES distribution are included in `test/data`, together with their flux, cross section and migration matrix files. They describe the experiments as they were planned when the files were written and should be checked against current designs before use.

File	Experiment
`NOvA.glb`	NOvA: ν_μ beam from NuMI, 25 kt detector at 812 km, ν_e appearance and ν_μ disappearance in neutrino and antineutrino mode
`NOvA_pre_post_smearing.glb`	the same, with efficiencies as pre- and post-smearing lists
`T2K.glb`	T2K: J-PARC beam to Super-Kamiokande (22.5 kt) at 295 km
`T2HK.glb`	T2HK: upgraded beam to a 500 kt water Cherenkov detector at 295 km
`NFstandard.glb`	neutrino factory: 50 GeV muons, 50 kt magnetised iron detector at 3000 km
`NFvar.glb`	the same with the AEDL variables `BASELINE` and `emax`
`NF_hR_lT.glb`	neutrino factory with high energy resolution and low threshold (variables `BASELINE`, `emax`)
`BB_350.glb`	beta beam with $\gamma = 350$, 500 kt water Cherenkov detector at 730 km, with migration matrices from a detector simulation
`BBvar_WC.glb`	beta beam with variable γ (`gammafactor`, `baselinefactor`, `EXP_FACTOR`)
`Reactor1.glb`, `Reactor2.glb`	reactor experiments with 20 and 100 t detectors at 1.7 km
`D-Chooz_near.glb`, `D-Chooz_far.glb`	Double Chooz near (0.1 km) and far (1.05 km) detectors, for a near–far analysis with a user-defined χ^2 (`examples/example5.jl`)
`Simple.glb`	the example of Chapter 10

Cross sections: `XCC.dat` (charged current), `XNC.dat` (neutral current), `XQE.dat` (quasi-elastic), `XCCreactor.dat` (inverse beta decay). Fluxes: `NOvAplus.dat`, `NOvAminus.dat`, `JHFplus.dat`, `JHFminus.dat`, `Reactor.dat`, `BB350flux.dat`.

Appendix B

Examples

The directory `examples` contains

`tour.jl`	the tour of Chapter 2, with the high-level API;
`example-tour.jl`	the same with the `glb*` functions, a line-by-line translation of `example-tour.c`;
`example1.jl`	the $\sin^2 2\theta_{13}$–δ_{CP} correlation with systematics only;
`example2.jl`	projections onto the $\sin^2 2\theta_{13}$ axis;
`example3.jl`	locating the sign degeneracy and its shape;
`example4.jl`	the θ_{13} sensitivity with statistics, systematics, correlations and degeneracies;
`example5.jl`	a reactor experiment with near and far detectors (Double Chooz) and user-defined χ^2 functions;
`example6.jl`	a user-defined probability engine: wave packet decoherence in a reactor experiment.

`examples/README.md` compares their output with the C examples.

Appendix C

Building this manual

The manual is in `docs/manual`. Its figures are drawn by pgfplots from tables computed by GLoBES.jl. To rebuild everything:

```
cd docs/manual
make            # runs figures/make_figures.jl and examples/tour.jl, then lualatex
```

`make figures` recomputes only the data, `make pdf` only typesets.

Bibliography

[1] P. Huber, M. Lindner and W. Winter, *Simulation of long-baseline neutrino oscillation experiments with GLoBES*, Comput. Phys. Commun. **167** (2005) 195, hep-ph/0407333.

[2] P. Huber, J. Kopp, M. Lindner, M. Rolinec and W. Winter, *New features in the simulation of neutrino oscillation experiments with GLoBES 3.0*, Comput. Phys. Commun. **177** (2007) 432, hep-ph/0701187.

[3] P. Huber, J. Kopp, M. Lindner, M. Rolinec and W. Winter, *GLoBES — General Long Baseline Experiment Simulator, User's Manual*, version 3.0.8, `https://www.mpi-hd.mpg.de/personalhomes/globes/documentation/globes-manual-3.0.8.pdf`.

[4] J. Kopp, *Efficient numerical diagonalization of hermitian* 3×3 *matrices*, Int. J. Mod. Phys. C **19** (2008) 523, physics/0610206.

[5] A. M. Dziewonski and D. L. Anderson, *Preliminary reference Earth model*, Phys. Earth Planet. Interiors **25** (1981) 297.

[6] F. D. Stacey, *Physics of the Earth*, 2nd ed., Wiley (1977).

[7] G. L. Fogli, E. Lisi, A. Marrone, D. Montanino and A. Palazzo, *Getting the most from the statistical analysis of solar neutrino oscillations*, Phys. Rev. D **66** (2002) 053010, hep-ph/0206162.

[8] P. Huber, M. Lindner and W. Winter, *Superbeams versus neutrino factories*, Nucl. Phys. B **645** (2002) 3, hep-ph/0204352.

[9] V. Barger, D. Marfatia and K. Whisnant, *Breaking eight-fold degeneracies in neutrino CP violation, mixing, and mass hierarchy*, Phys. Rev. D **65** (2002) 073023, hep-ph/0112119.

[10] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, *Numerical Recipes in C*, 2nd ed., Cambridge University Press (1992), Sec. 10.5.

[11] F. N. Fritsch and R. E. Carlson, *Monotone piecewise cubic interpolation*, SIAM J. Numer. Anal. **17** (1980) 238; F. N. Fritsch and J. Butland, *A method for constructing local monotone piecewise cubic interpolants*, SIAM J. Sci. Stat. Comput. **5** (1984) 300.

[12] E. K. Akhmedov, R. Johansson, M. Lindner, T. Ohlsson and T. Schwetz, *Series expansions for three-flavor neutrino oscillation probabilities in matter*, JHEP **04** (2004) 078, hep-ph/0402175.

[13] P. Huber and W. Winter, *Neutrino factories and the "magic" baseline*, Phys. Rev. D **68** (2003) 037301, hep-ph/0301257.